



Requirement & Design Specification

Anti-Fraud Attendance System (AFAS)

Subject: SWD392

Version: 1.0

– Hanoi, August 2024 –

Record of Changes

Version	Date	A* M, D	In charge	Change Description

*A - Added M - Modified D - Deleted

Contents

I. Requirement Specification.....	3
I.1 Problem description.....	3
I.2 Major Features.....	5
I.2.1 Features for Students.....	5
I.3 System context.....	6
I.4 Nonfunction Requirements.....	7
I.5 Functional requirements.....	8
I.5.1 Use case diagrams.....	8
I.5.2 Use case descriptions.....	10
I.5.3 Activity diagrams.....	41

I.6 Data Requirements.....	44
II. Analysis models.....	50
II.1 Interaction diagrams.....	50
II.2 State diagram.....	58
III. Design specification.....	59
III.1 Integrated Communication Diagrams.....	59
III.2 System High-Level Design.....	60
III.3 Component and Package Diagram.....	62
III.4 Detail Design.....	62
III.5 Database Design.....	63
IV. Implementation.....	65
IV.1 Map architecture to the structure of the project.....	65
IV.2 Map Class Diagram and Interaction Diagram to Code.....	66
V. Applying Alternative Architecture Patterns.....	72
V.1 Applying the service-oriented architecture.....	72
V.2 Applying Service Discovery Pattern in the service-oriented architecture.....	74

I. Requirement Specification

I.1 Problem description

Purpose: The purpose of this project is to develop an Anti-Fraud Attendance Management System for universities and educational institutions. The system aims to automate the attendance process while preventing common fraudulent behaviors such as proxy attendance, QR code sharing, and remote check-ins outside the classroom area.

1. The system is designed for approximately 8,000 students and supports three main roles: Students, Lecturers, and Administrators. The system provides mobile/web interfaces for students and web portals for lecturers and administrators.
2. Students can log into the system using their student accounts to perform attendance activities. During each class session, lecturers generate a dynamic QR code that changes automatically every 10 seconds. Students use the mobile application to scan the QR code and submit attendance information. Along with the attendance request, the system automatically collects the student's GPS location and device identifier (Device ID/UUID) to verify authenticity.
3. To prevent fraudulent attendance activities, the system applies multiple anti-fraud mechanisms. The first layer uses Dynamic QR Codes with temporary tokens and

timestamps. The server only accepts valid QR tokens within a limited time window of approximately 10–15 seconds. This prevents students from taking screenshots of the QR code and sending them to others outside the classroom.

4. The second layer applies Geofencing verification. When attendance data is received, the server calculates the distance between the student's current GPS coordinates and the classroom coordinates using distance calculation algorithms such as the Haversine formula. If the distance exceeds the allowed threshold (for example, 20 meters), the attendance request will be rejected or marked as suspicious.
5. The third layer focuses on preventing students from using another person's mobile device for attendance. The system may integrate biometric verification mechanisms such as facial recognition or other identity verification methods.
6. Lecturers can manage their assigned classes, activate attendance sessions, monitor attendance status in real-time, manually adjust attendance results when necessary, and export attendance reports to Excel files.
7. Administrators are responsible for managing system data such as student accounts, lecturer accounts, courses, classrooms, and GPS configurations for each classroom.
8. The system also focuses on non-functional requirements such as concurrency, usability, performance, and location accuracy. During the first few minutes of class, the system must support a large number of simultaneous attendance requests efficiently.

I.2 Major Features

I.2.1 Features for Students

- F01: Student Authentication: Login using student ID and password, secure authentication and session management.
- F02: QR Attendance: Scan dynamic QR code, submit attendance request, automatically send GPS location and device ID.
- F03: Attendance Verification: Verify attendance status, detect invalid or expired QR codes, detect location violations outside classroom area.
- F04: Attendance History: View attendance history, view attendance status (Present, Late, Absent), track total attendance statistics.

- F05: Profile Management: View personal information, update profile information, manage registered device information.

1.2.2 Features for Lecturers

- F06: Class Management: View assigned classes, view student lists for each class.
- F07: Dynamic QR Attendance Session: Create attendance sessions, generate dynamic QR codes, automatically refresh QR token every 10 seconds.
- F08: Real-Time Attendance Monitoring: Monitor attendance progress in real time, receive instant attendance updates through Socket/Polling.
- F09: Attendance Adjustment: Modify attendance results manually, approve special attendance cases.
- F10: Attendance Reporting: Export attendance reports to Excel, view attendance statistics for each class.

1.2.3 Features for Administrators

- F11: User Management: Add, edit, delete student accounts, add, edit, delete lecturer accounts, assign permissions and roles.
- F12: Course and Class Management: Manage subjects and course information, manage class schedules and classroom assignments.
- F13: Classroom GPS Configuration: Configure GPS coordinates for classrooms, configure allowed attendance radius.
- F14: System Configuration: Configure attendance rules, configure QR expiration time, configure anti-fraud settings.
- F15: Attendance Statistics and Monitoring: View attendance reports, detect suspicious attendance activities, monitor system performance and logs.

I.3 System context

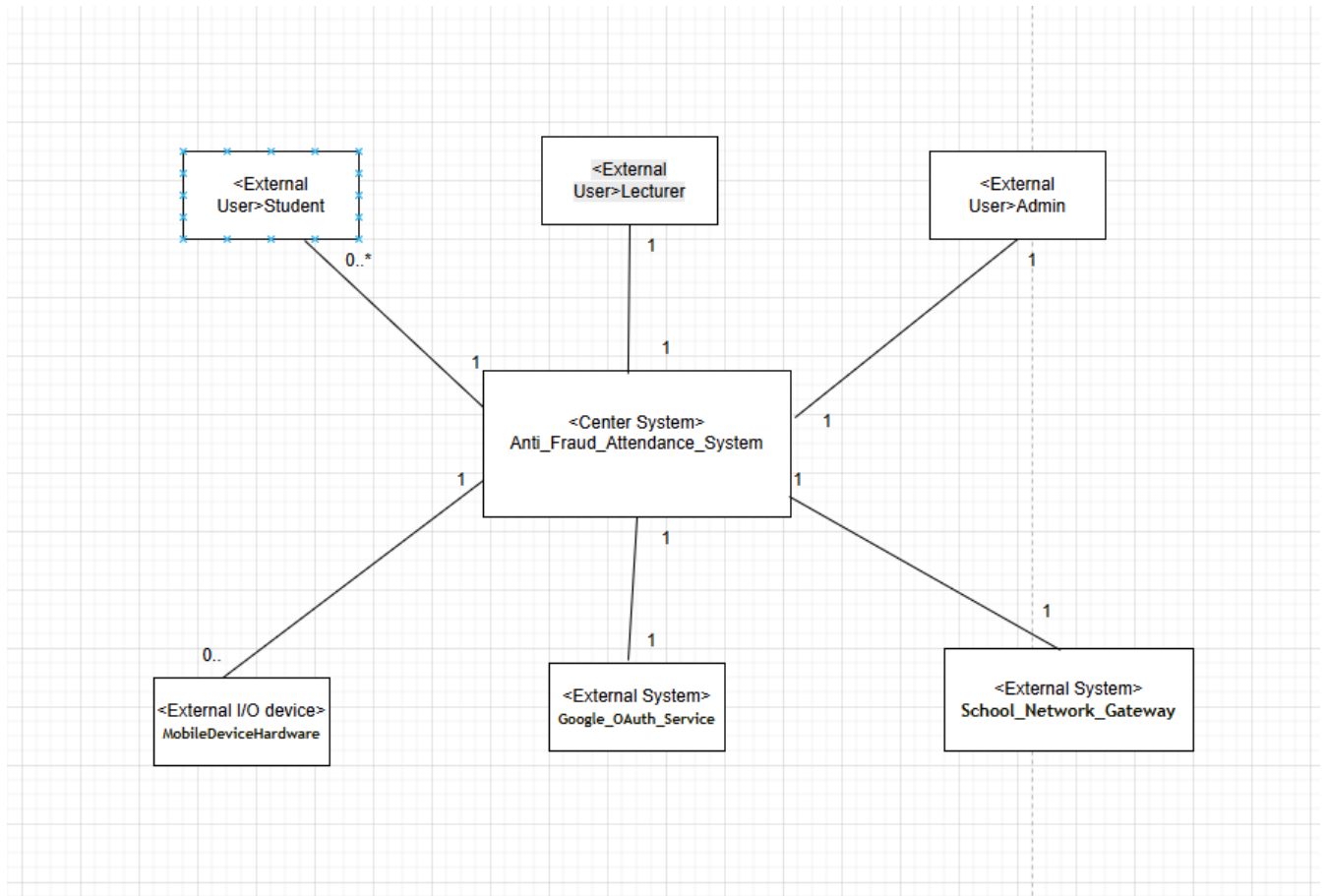


Figure I-1 System context is modeled by a class diagram

Figure 1-1 shows the system context of the Anti_Fraud_Attendance_System, illustrating the interactions between the system and its various entities including external users and external systems.

External Users

Student

The primary end-user of the system who can securely perform attendance check-in, scan dynamic QR codes, complete biometric verification, and view attendance history.

Lecturer

Responsible for managing attendance sessions, generating dynamic QR codes, monitoring real-time attendance results, handling suspicious attendance cases, and exporting attendance reports.

Administrator

Responsible for managing user accounts, configuring attendance policies, managing classroom and geofence settings, and reviewing fraud detection logs and system activities.

External Systems

Mobile Device Hardware

Provides device-level services required for anti-fraud attendance verification, including GPS location, Device UUID identification, camera access, and biometric authentication such as Face ID or fingerprint verification.

Google Authentication Service

Provides authentication and login services using institutional or Google accounts.

School Network Gateway

Supports verification of campus network information and assists in detecting suspicious attendance requests originating outside the school network environment.

Notification Service

Sends attendance-related notifications to students and lecturers, including attendance confirmations, suspicious activity warnings, and attendance session announcements.

I.4 Non-functional Requirements

NF-01 Maintainability

The system should be easy to maintain, update, and extend in the future. Source code must be organized using a modular structure with clear separation between authentication, attendance processing, anti-fraud validation, and reporting modules.

System documentation should clearly describe:

1. Component responsibilities
2. Module relationships
3. Database structure
4. Validation workflows

This helps developers quickly understand the system and implement future changes without affecting existing features.

NF-02 Usability

The system should provide a simple and user-friendly interface for both students and lecturers. Attendance check-in must be fast and easy to perform during live classroom sessions.

The system should:

1. Allow students to complete attendance check-in within 30 seconds
2. Display clear messages for successful, rejected, or suspicious attendance attempts
3. Provide lecturers with real-time attendance updates
4. Minimize user errors during QR scanning and verification

Target usability metrics:

1. Attendance success rate: at least 95%
 2. User satisfaction score: at least 85%
 3. Failed attendance attempts caused by UI confusion: less than 5%
-

NF-03 Portability

The system should support multiple devices and platforms commonly used in university environments.

The mobile attendance application should support:

1. Android and iOS devices
2. Different screen sizes and resolutions
3. Modern mobile browsers and operating systems
4. The lecturer dashboard should support: Latest versions of Chrome, Edge, Firefox, and Safari

The interface should use responsive design to ensure usability across phones, tablets, and desktop devices.

NF-04 Performance

The system should provide fast and stable attendance processing during peak classroom usage.

The system should:

1. Generate and refresh dynamic QR codes in real time
2. Process attendance validation quickly
3. Support simultaneous attendance submissions from multiple students

Performance targets:

1. QR validation response time: less than 3 seconds
 2. Support at least 300 concurrent attendance requests
 3. Real-time lecturer dashboard updates within 2 seconds
 4. Performance degradation under stress conditions should remain below 20%
-

NF-05 Security

The system should protect attendance data and prevent fraudulent attendance activities.

The system should:

1. Use encrypted communication (HTTPS)
2. Apply biometric verification and trusted device validation
3. Prevent duplicate attendance submissions
4. Detect suspicious attendance attempts such as QR sharing or remote check-in
5. Restrict students to viewing only their own attendance records

All suspicious activities must be recorded for auditing and administrative review.

NF-06 Reliability

The system should maintain stable attendance operations during classroom sessions.

The system should:

1. Prevent data loss during attendance submission
2. Recover gracefully from temporary network interruptions
3. Ensure attendance records are stored correctly even during high traffic periods

Attendance data accuracy is critical because it directly affects academic records and student participation tracking.

I.5 Functional requirements

I.5.1 Use case diagrams

The functional requirements are mapped to three main use case diagrams representing the Student, Lecturer, and Admin subsystems.

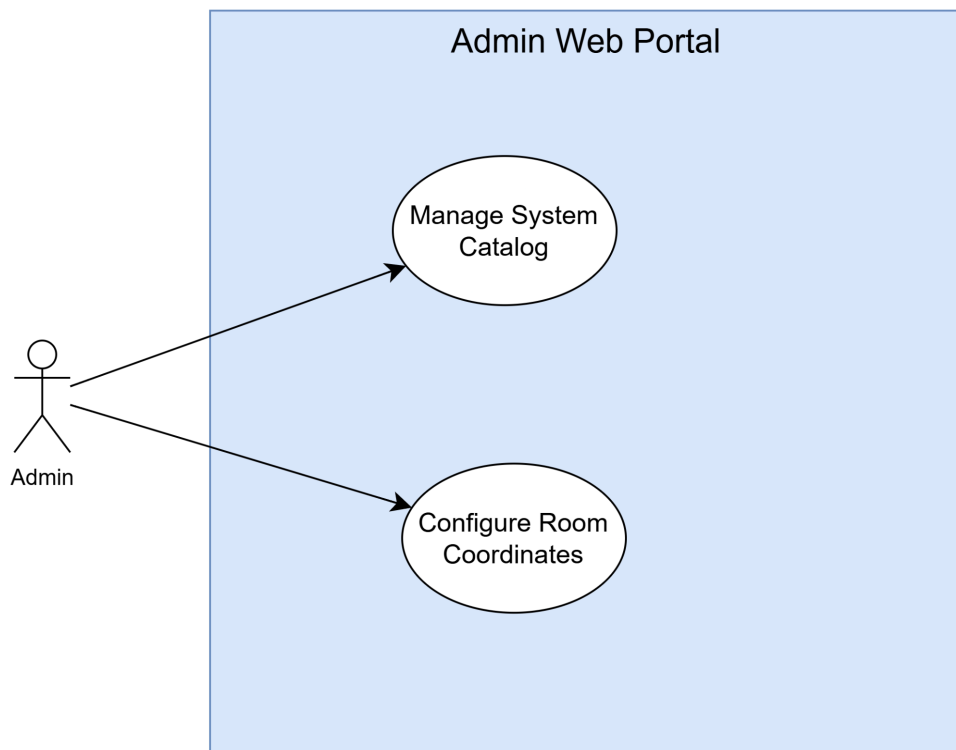


Figure I-2 Overview Use Case Diagram of Admin Site

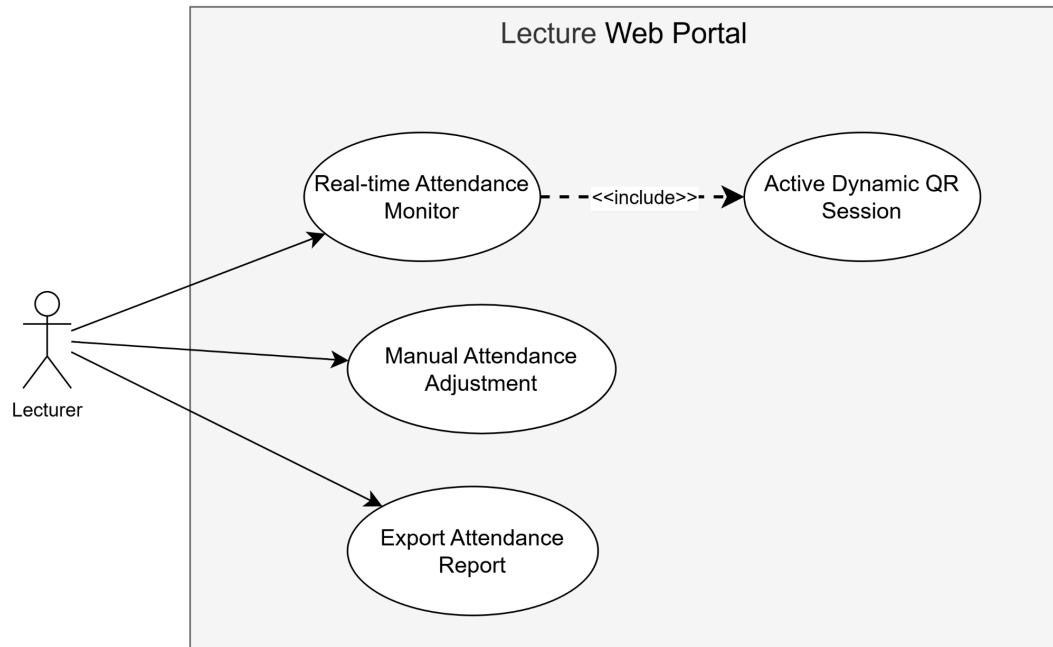


Figure I-3 Overview Use Case Diagram of Lecturer Site

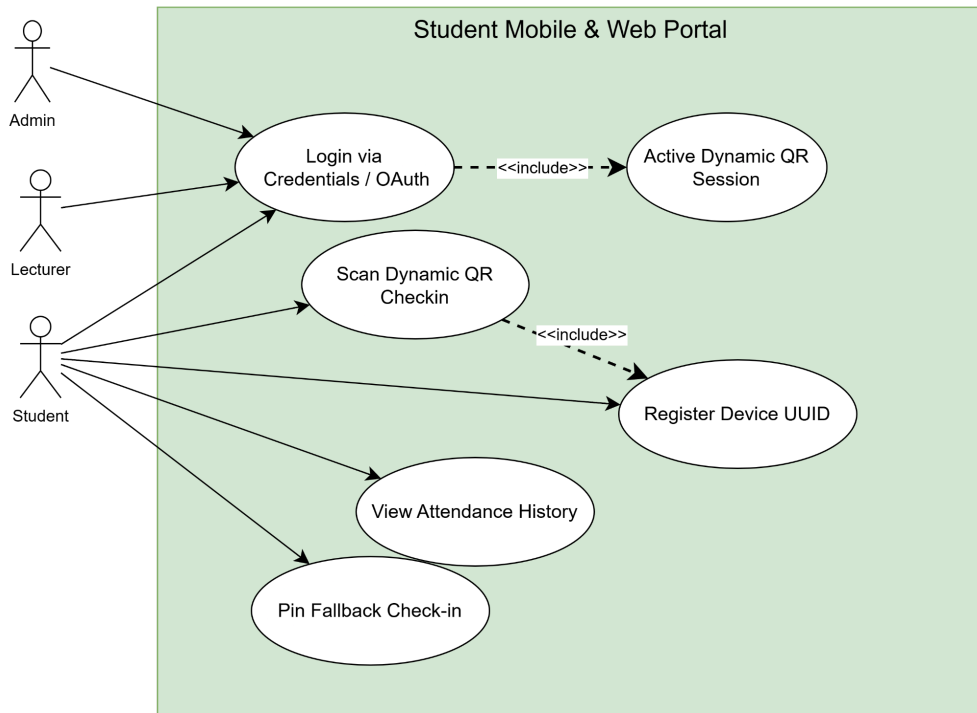


Figure I-4 Detailed Use Case Diagram for Student Site

I.5.2 Use case descriptions

Tables 1-1 to 1-6 describe the use cases: Manage Products, Add Product, Edit Product, Delete Product, Disable News, and Enable News. The event flows of these use cases can be modeled using activity diagrams. For example, Figures 1-12 and 1-13 are activity diagrams modeling the event flows of the Manage Products and Add Product use cases.

Table I-1 LOGIN VIA CREDENTIALS OR GOOGLE OAUTH

ID and Name:	UC01: Login via Credentials or Google OAuth		
Created By:	ThinhTH	Date Created:	27/05/2026
Primary Actor:	Student, Lecturer, Admin	Secondary Actors:	
Description:	Allows any system user to securely authenticate and access their respective portal using either MSSV/Password or Google OAuth (FPT Mail).		
Trigger:	The user opens the mobile application or visits the web portal.		
Preconditions:	PRE-1: • For Credentials Login: The user account must be pre-created and active within the system database. • For Google OAuth: The user must possess a valid institutional email address belonging to the FPT education group ecosystem. PRE-2: The user's device must have an active internet connection to communicate with the application backend server and the external Google Gateway.		

Postconditions:	POST-1 Success: User is authenticated, a secure token (JWT) is issued, and the user is redirected to their dashboard. POST-2 Failure: Authentication fails, access is denied, and no session is created.		
Normal Flow:	1. User selects login method: "Credentials" or "Google OAuth". 2. If Credentials: User inputs MSSV/Username and Password, then submits. (See A2.1) 3. If Google OAuth: User clicks Google Login, authenticates through Google Gateway, and returns FPT email. 4. Server validates credentials against Accounts database or verifies the Google OAuth token. 5. Server generates a secure JWT token containing the user's role. 6. Server redirects user to their corresponding homepage.		
Alternative Flows:	A2.1 User forgets password: User selects "Forgot Password", inputs registered email, receives reset link, and updates password.		
Exceptions:	E4.1 Invalid credentials: Server returns an error message: "Invalid username or password". E4.2 Non-school email: If Google OAuth returns a non-FPT email (not ending in @fpt.edu.vn), server denies login.		
Priority:	High		
Business Rules:	BR-01: Passwords must be hashed using bcrypt on the server. BR-02: Email domain must end with @fpt.edu.vn or @fe.edu.vn for Google OAuth		

Table I-2 REGISTER DEVICE UUID

ID and Name:	UC02: Register Device UUID		
Created By:	ThinhTH	Date Created:	27/05/2026

Primary Actor:	Student	Secondary Actors:	
Description:	Binds a student's account to a single unique physical mobile device upon first login to prevent multiple students from using one phone to check in for others.		
Trigger:	The student logs into the mobile application for the first time or on a new device.		
Preconditions:	Student is authenticated (UC01) and does not have an active device binding or has requested a device reset.		
Postconditions:	POST-1 Success: Student's DeviceUUID is recorded in the Students table, linking the account to the phone. POST-2 Failure: Device is not linked, and student cannot proceed to scan QR.		
Normal Flow:	1. Student logs into the AFAS App on a mobile device. 2. App checks if the student's record in students.device_uuid is null. 3. Since it is null, the App automatically extracts the hardware DeviceUUID of the phone. 4. App displays a message informing the student that this phone will be registered as their primary check-in device. 5. Student confirms the binding. 6. Server saves the DeviceUUID to the student's profile.		
Alternative Flows:	A3.1 Student requests device reset (Self-Service): 1. Student logs in on a new device and is notified that a device is already bound. 2. Student clicks "Request Device Reset". 3. System sends an OTP code to the student's registered FPT email. 4. Student enters the correct OTP on the screen. 5. System clears the old device_uuid, extracts the new device's UUID, and saves it.		

Exceptions:	E4.1 Invalid OTP: If the student enters an incorrect OTP 3 times during reset, the reset process is locked for 24 hours.
Priority:	High
Business Rules:	BR-01: One student account can only be linked to one DeviceUUID at a time.

Table I-3 Use case description for Edit product

ID and Name:	UC03: Scan Dynamic QR Check-in		
Created By:	CuongPH	Date Created:	29/05/2026
Primary Actor:	Student	Secondary Actors:	Non
Description:	Allows students to check in for a class attendance session by scanning a dynamic QR code displayed by the lecturer. The system validates biometric authentication, geolocation, registered device identity, and QR token validity before recording attendance.		
Trigger:	The student selects the “Scan QR Check-in” feature from the application dashboard.		
Preconditions:	- Student is authenticated successfully (UC01). - Student device has already been registered and linked to the account (UC02) .- An attendance session is currently active .- Camera and GPS permissions are enabled on the device.		
Postconditions:	POST-1 Success: An AttendanceRecord is created with status Present or Late and the lecturer dashboard is updated in real time. POST-2 Failure: Check-in request is rejected and an appropriate error message is returned to the student.		

Normal Flow:	1. Student opens the “Secure Check-in” feature. 2. The system requests biometric verification to confirm the student identity. 3. Student successfully verifies identity on the device. 4. Student scans the QR code displayed in the classroom. 5. The system validates that the QR code belongs to an active attendance session. 6. The system verifies that the student is physically located within the allowed classroom area. 7. The system verifies that the attendance request is submitted from the student’s registered device. 8. The system checks whether the student has already checked in for the session. 9. The system determines the attendance status as Present or Late based on the check-in time. 10. The system records the attendance result. 11. The system updates the lecturer attendance dashboard in real time. 12. The student receives a successful attendance confirmation message.
Alternative Flows:	
Exceptions:	A3.1 Biometric Verification Unavailable: If biometric verification is unavailable on the device, the system requests an additional selfie verification step before allowing attendance submission. A3.2 Late Check-in: If the student performs attendance after the allowed attendance period, the system records the attendance status as Late.
Priority:	High
Business Rules:	BR-01: QR codes must expire automatically after a short period of time. BR-02: A student may only submit one attendance record per session. BR-03: Attendance is marked Present or Late based on configured attendance thresholds. BR-04: Attendance requests failing anti-fraud validation must be rejected or flagged. BR-05: Temporary biometric verification data must not be permanently stored.

Table I-4 Use case description for Delete product

ID and Name:	UC04: View Attendance History		
Created By:	CuongPH	Date Created:	29/05/2026
Primary Actor:	Student	Secondary Actors:	Non
Description:	Allows students to review their attendance records, attendance statistics, and attendance validation results for enrolled courses.		
Normalflow	1. Student opens the “Attendance History” feature. 2. The system displays a list of enrolled courses or class sections. 3. Student selects a course or class section. 4. The system displays attendance records for the selected course. 5. The system displays attendance details including attendance date, attendance status, and check-in time. 6. The system displays attendance statistics such as Present, Late, Absent, and attendance percentage. 7. The system highlights attendance records that were rejected or flagged as suspicious.		
Alternative Flows:	None		
Exceptions:	E4.1 Unauthorized Access: The system blocks access if the student session is invalid or expired. E4.2 Attendance Data Unavailable: The system displays an error message if attendance records cannot be loaded temporarily		
Priority:	High		
Business Rules:	BR-01: Geofence formula must use the Haversine method on the server. BR-02: Face selfies captured during check-in must be deleted immediately after verification. BR-03: A student is marked Present if check-in occurs within 15 minutes of the session start time; Late if after that threshold but before session end.		

	BR-04: Only one AttendanceRecord per student per session is allowed. Duplicate submissions return the existing result.
--	--

Table I-5 Use case View attendance history & Submit appeal

ID and Name:	UC-05 VIEW ATTENDANCE HISTORY & SUBMIT APPEAL		
Created By:	TrungNT	Date Created:	28/05/2024
Primary Actor:	Student	Secondary Actors:	
Description:	Allows students to view their attendance records for all enrolled classes. If a session is marked as Fraud_Declined (due to GPS or Device mismatch), the student can submit an appeal with evidence (e.g., a photo in class) for the Lecturer to review.		
Trigger:	The student selects the "Attendance History" tab from the mobile application's main menu.		
Preconditions:	PRE-1: Student is authenticated (UC01). PRE-2: Student is enrolled in at least one active class section.		
Postconditions:	POST-1: Student successfully views their attendance status. POST-2: (If appealed) An appeal record is created and its status is marked as Pending_Review.		
Normal Flow:	1. Student navigates to the "Attendance History" tab on the mobile app. 2. System fetches all past and current sessions from AttendanceRecord. 3. System displays a list of sessions, color-coded by status (Present, Late, Absent, Fraud_Declined). 4. Student selects a specific session to view details (Time checked in, distance from classroom, device used).		

Alternative Flows:	
Exceptions:	E4.1 Unauthorized Access: The system blocks access if the student session is invalid or expired. E4.2 Attendance Data Unavailable: The system displays an error message if attendance records cannot be loaded temporarily
Priority:	Medium
Business Rules:	BR-01: Students may only view their own attendance records. BR-02: Attendance statistics must be calculated automatically by the system. BR-03: Suspicious or rejected attendance records must be clearly identified in the history view.

Table I-6 Use case Scan QR & Submit attendance

ID and Name:	UC-06 SCAN QR & SUBMIT ATTENDANCE		
Created By:	Trungnt	Date Created:	28/05/2026
Primary Actor:	Student	Secondary Actors:	
Description:	The core function where a student uses the mobile app to scan the Lecturer's dynamic QR code. The app bundles the QR token, current GPS coordinates, and Device UUID, then sends them to the server to verify presence and prevent fraud.		
Trigger:	The student taps the "Scan QR" button on the mobile application to activate the camera and point it at the lecturer's screen.		
Preconditions:	PRE-1: Student is authenticated. PRE-2: Student has successfully bound their Device UUID (UC02). PRE-3: Lecturer has activated an attendance session (UC07).		
Postconditions:	POST-1 (Success): AttendanceRecord is marked as Present or Late. POST-2 (Fraud Detected): AttendanceRecord is marked as Fraud_Declined and the incident is logged.		

Normal Flow:	1. Student opens the QR Scanner in the AFAS mobile app. 2. App requests and retrieves high-accuracy GPS coordinates from the OS. 3. Student scans the Dynamic QR code displayed on the Lecturer's screen. 4. App extracts the SessionToken from the QR. 5. App sends a POST request with: SessionToken, Latitude, Longitude, and DeviceUUID. 6. Server validates the SessionToken timestamp (must be within the 15-second window). 7. Server verifies the DeviceUUID matches the database record. 8. Server calculates the distance between student's GPS and classroom GPS (using Haversine formula). 9. Server marks attendance as Present and returns a success message. 10. App displays a green "Check-in Successful" animation.
Alternative Flows:	A1.1: Late Check-in At Step 9, if the scan occurs after the strict "Present" window but within the "Late" window, system marks status as Late.
Exceptions:	E6.1: Token Expired Server rejects the token if it's older than 15 seconds. App shows: "QR Code expired, please scan the newest one." E7.1: Device Mismatch (Fraud Layer 3) If DeviceUUID does not match, server returns HTTP 403. Status -> Fraud_Declined. E8.1: GPS Out of Bounds (Fraud Layer 2)

	If calculated distance > AllowedRadius (e.g., 20m), server rejects attendance. Status -> Fraud_Declined.
Priority:	Critical
Business Rules:	BR-01: Geofencing validation is mandatory. If OS location services are disabled, the app blocks the scanner. BR-02: Fraud_Declined records cannot be overwritten by subsequent scans in the same session.

Table I-7 Use case Activate Dynamic QR attendance session

ID and Name:	UC07: ACTIVATE DYNAMIC QR ATTENDANCE SESSION		
Created By:	Trungnt	Date Created:	28/05/2026
Primary Actor:	Lecturer	Secondary Actors:	
Description:	Allows the Lecturer to start an attendance session on the Web Portal. The system generates a cryptographic token encoded into a QR code, which automatically refreshes on the screen every 10 seconds to prevent screenshot sharing.		
Trigger:	The lecturer selects a scheduled class session on the Web Portal and clicks the "Start Attendance" button.		
Preconditions:	PRE-1: Lecturer is authenticated. PRE-2: Lecturer is the owner of the class section.		
Postconditions:	POST-1: A new Session record is created (or updated to Active). A WebSocket connection is established to stream real-time scan data.		

Normal Flow:	1. Lecturer navigates to the Class Dashboard and clicks "Start Attendance". 2. System initializes the session and sets status to Active. 3. Server generates the first SessionToken (hashed with a timestamp) and sends it to the frontend. 4. Web Portal renders the Token into a large QR code. 5. A background timer starts on the Web Portal. Every 10 seconds, it requests a new SessionToken via WebSocket/API. 6. Server generates and returns the new token. 7. Web Portal updates the QR code seamlessly.
Alternative Flows:	A1.1: End Session Lecturer clicks "Close Attendance". Server invalidates all active tokens and marks the session as Closed. Any subsequent scans are rejected.
Exceptions:	E3.1: Network Disconnect If the Web Portal loses internet, the UI blurs the QR code and shows a "Reconnecting..." overlay to prevent students from scanning stale tokens.
Priority:	High
Business Rules:	BR-01: QR tokens must have a strict TTL (Time-To-Live) of 15 seconds on the server side to account for a 5-second network latency buffer.

Table I-8 USE CASE MANUAL ATTENDANCE ADJUSTMENT

ID and Name:	UC08: MANUAL ATTENDANCE ADJUSTMENT		
Created By:	VuNT	Date Created:	27/05/2026
Primary Actor:	Lecturer	Secondary Actors:	

Description:	Allows the lecturer to manually change a student's check-in status (e.g., overriding a fake GPS fraud flag if there is a hardware error, or marking an absent student manually).
Trigger:	Lecturer selects a student name from the list and clicks "Adjust Status".
Preconditions:	Lecturer is authenticated (UC01).
Postconditions:	Student status is updated in the database and audit logged.
Normal Flow:	1. Lecturer views the student roster for the active/past session. 2. Lecturer clicks on a specific student tile and selects "Adjust Status". 3. System displays a modal with status options: Present, Late, Absent, Fraud_Declined. 4. Lecturer selects the new status and enters a reason (e.g., "GPS device hardware error"). 5. Lecturer clicks "Save". 6. Server updates AttendanceRecord status and verification_mode is set to 'Manual' 7. Server logs the lecturer's action in SystemLogs.
Alternative Flows:	None.
Exceptions:	E5.1 Missing reason: If the lecturer changes status without inputting a mandatory reason, the system prompts them to write a reason before saving.
Priority:	High

Business Rules:	BR-01: All manual overrides must record the modifier's ID and a mandatory explanation.
-----------------	--

Table I-9 USE CASE EXPORT ATTENDANCE REPORT

ID and Name:	UC09: EXPORT ATTENDANCE REPORT		
Created By:	VuNT	Date Created:	27/05/2026
Primary Actor:	Lecturer	Secondary Actors:	
Description:	Exports the attendance statistics sheet for a specific class section or semester into an Excel format (.xlsx) for grading and academic records.		
Trigger:	The lecturer clicks the "Export Excel" button on the class details screen.		
Preconditions:	Lecturer is logged in (UC01).		
Postconditions:	Excel file is downloaded to the lecturer's local computer.		
Normal Flow:	1. Lecturer navigates to class detail view. 2. Lecturer clicks "Export Excel". 3. System compiles all session records of that class from attendance_records and class_section_students. 4. System formats the data into a grid containing student info, date of sessions, check-in mode, and aggregate attendance percentage. 5. System generates a .xlsx spreadsheet download stream. 6. Lecturer saves the spreadsheet locally.		

Alternative Flows:	None.
Exceptions:	None.
Priority:	Medium
Business Rules:	None.

Table I-10 USE CASE MANAGE SYSTEM CATALOG

ID and Name:	UC10: MANAGE SYSTEM CATALOG		
Created By:	VuNT	Date Created:	27/05/2026
Primary Actor:	Admin	Secondary Actors:	Email Server
Description:	Allows administrative staff to create, update, or delete system raw catalog records: User accounts (Students/Lecturers), Subjects, and Class Sections.		
Trigger:	Admin clicks on any catalog link in the Admin Portal menu.		
Preconditions:	Admin is logged in (UC01).		
Postconditions:	Catalog data is updated in PostgreSQL.		

Normal Flow:	1. Admin logs into the Admin Portal. 2. Admin clicks on a catalog menu option (e.g., "Students", "Subjects"). 3. System displays a grid with search/add/edit/delete actions. 4. Admin inputs new student details (Student ID, Full Name, Email) and submits. 5. Server validates input and writes record to Accounts and Students tables.
Alternative Flows:	A4.1 Batch Import: Admin uploads a .csv or .xlsx spreadsheet containing hundreds of student/subject rows. System parses, runs validation, and performs batch writes to the DB.
Exceptions:	E5.1 Duplicate ID: If Admin attempts to add a student ID that already exists, system displays a validation error: "ID already exists".
Priority:	High
Business Rules:	None.

*Table I-11 USE CASE CONFIGURE ROOM COORDINATES &
ALLOWED RADIUS*

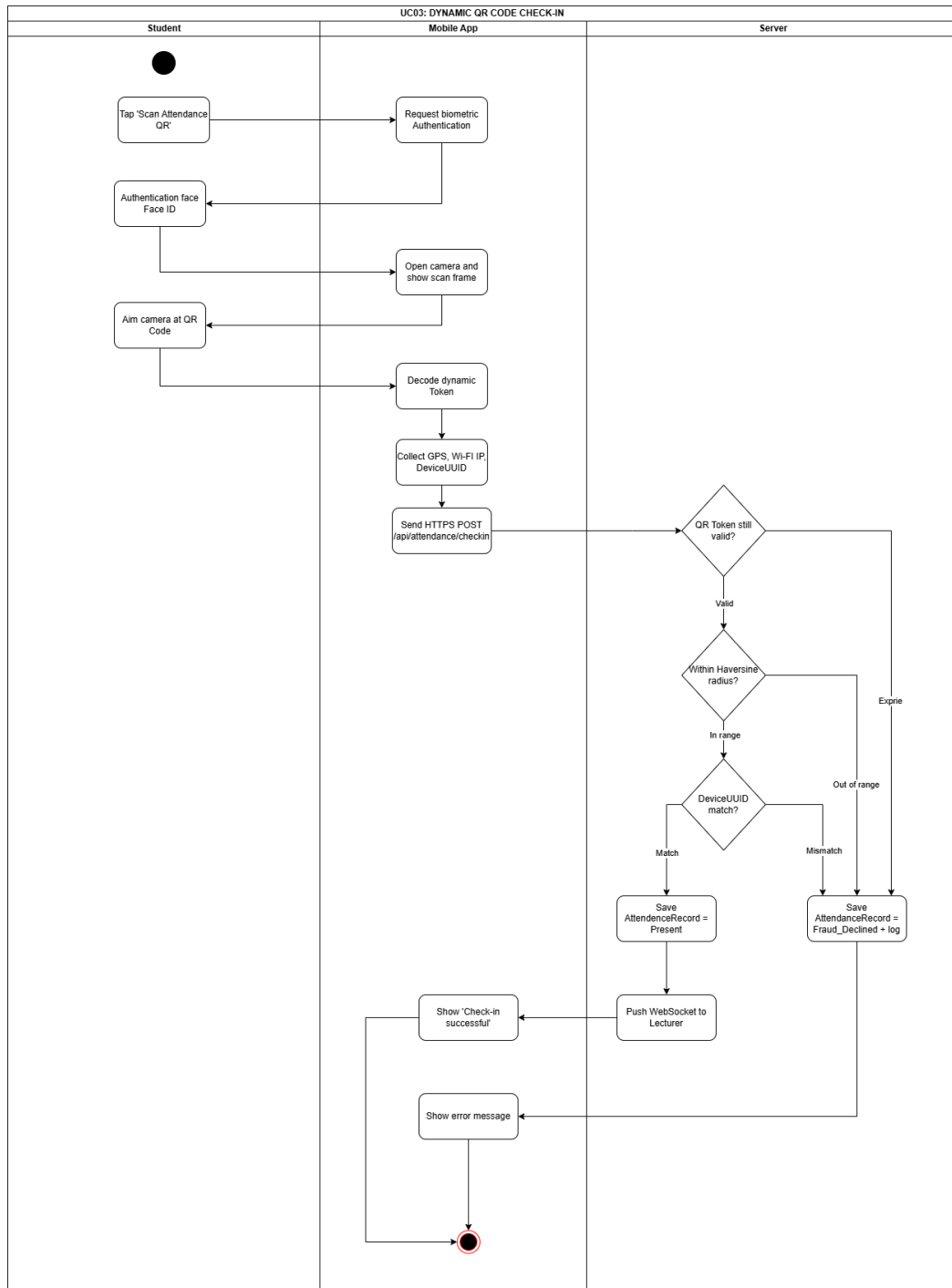
ID and Name:	UC11: CONFIGURE ROOM COORDINATES & ALLOWED RADIUS		
Created By:	VuNT	Date Created:	27/05/2026
Primary Actor:	Admin	Secondary Actors:	
Description:	Admin configures the exact Latitude, Longitude, and allowed geofence radius for classrooms on campus, which serves as the coordinates source for geofencing.		
Trigger:	Admin clicks "Room Management" on the dashboard.		

Preconditions:	Admin is logged in (UC01).
Postconditions:	Room geo-coordinates are updated in Rooms table.
Normal Flow:	1. Admin navigates to "Room Management". 2. System displays all physical classrooms on campus. 3. Admin selects a room (e.g., AL-L402) and clicks "Configure Geo". 4. System opens a configuration form with an integrated satellite map view. 5. Admin clicks on the exact classroom center point on the satellite map or manually inputs decimals into the Latitude and Longitude fields. 6. Admin enters the Allowed Radius parameter (e.g., 20 meters). 7. Admin clicks "Save Configuration". 8. Server verifies coordinate bounds, writes values to the Rooms table, and logs the administrative action.
Alternative Flows:	A5.1 On-site Mobile Calibration: Admin visits the room physically on-site, opens the Admin web view on a tablet, and clicks "Capture Current GPS". The tablet's high-precision hardware GPS coordinates are automatically populated.
Exceptions:	E8.1 Out-of-bounds Coordinates: If Admin inputs coordinates that are not within the university's bounding box, system prompts a warning to verify the number.
Priority:	High
Business Rules:	BR-01: The default AllowedRadius is 20 meters if no value is configured, compensating for normal indoor GPS hardware drift.

1.5.3 Activity diagrams

Figures 1-12 and 1-13 are activity diagrams modeling event flows of the use case Manage products and Add product.

UC03: DYNAMIC QR CODE CHECK-IN



UC06: SCAN QR & SUBMIT ATTENDANCE

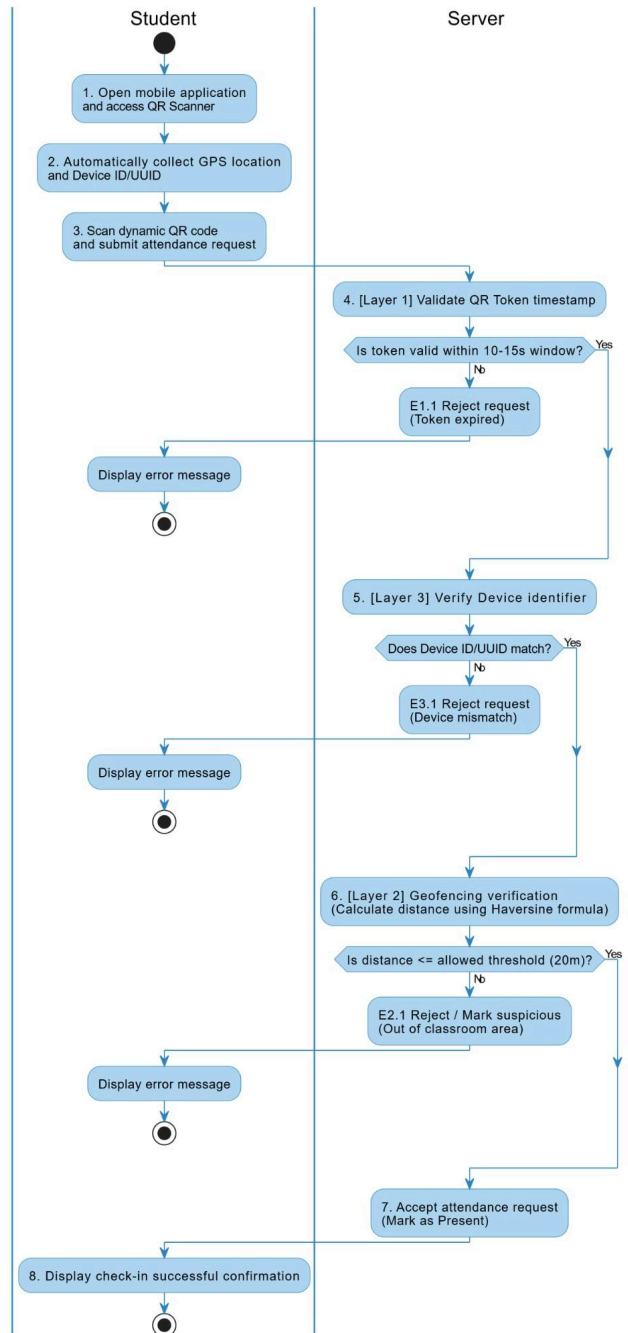


Figure I-12 Activity diagram modeling event flows of use case Manage product

I.6 Data Requirements

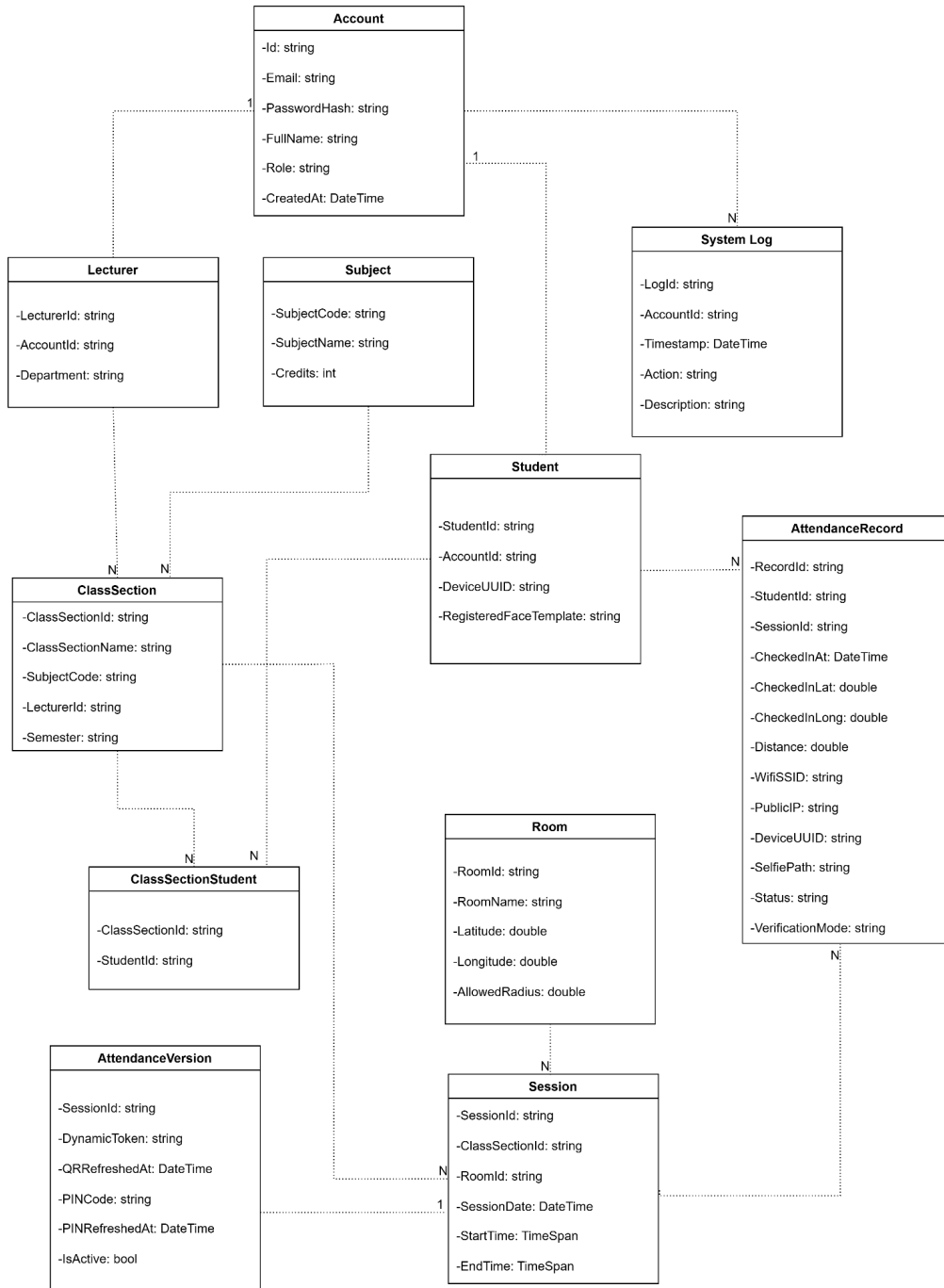


Figure I-14 Entity class diagram modeling data requirements

The class diagram in Figure 1.14 specifies the entity classes and the relationships between them, clearly representing the system's data requirements. Each entity class in the diagram represents a domain concept, relationships between entities are associations, aggregations, composition, or inheritances. To provide detailed information about these entity classes, they are all thoroughly described in Table 1-7, which is known as the data dictionary. This data dictionary serves as an explanatory tool, offering detailed descriptions of each class's attributes, data types, and relationships, helping the reader better understand the data structure and system requirements.

Table I-7 Data Description (Data dictionary)

Name	Data Type	Length	Description
Account			User accounts credential catalog
CatId	String	20	Product Category Identifier
CatName	String	50	Product Category Name
Description	String	500	Description of product category
Product			
ProId	String	20	Product Identify
ProName	String	50	Product Name
Measure	String	50	The unit of measurement used to quantify the product.
Description	String	Max	Description of product
Image	String	50	The visual representation of the product. It stores information about an image file that shows what the product looks like.
Supplier			
SupId	String	20	Supplier identifier
Name	String	50	Supplier Name
Address	String	500	Supplier Address

Phone	String	15	The contact telephone number for the supplier
Customer			
CusId	String	20	Customer Identifier

CusName	String	50	Customer Name
Phone	String	15	The contact telephone number for the customer
Email	String	50	The email address of the customer
Address	String	500	The physical address of the customer
Discount			
DisId	string	20	Discount Identifier
DisName	String	250	The name or description of a discount applied to orders or products.
DisAmount	Float		The monetary value or percentage of the discount applied to an order or product
IsPercentage	Boolean		It indicates whether the discount amount is represented as a percentage or a fixed monetary value
StartDate	DateTime		The date and time when a particular record or action becomes effective
EndDate	DateTime		The date and time when a particular record or action expires or is no longer effective
IsActive	Boolean		It indicates whether a particular discount is currently active and can be applied or not.
ShippingAddress			
AddId	String	20	Shipping address Identifier
CuId	String	20	Customer Identifier
Province	String	50	It is a part of the address and represents a specific administrative division within a country, such as a city, province, or region

District	String	50	It is a part of the address and represents a specific administrative division within provinces and cities
Commune	String	50	It is an address represents a smaller administrative division within a district
AddressLine	String	100	It typically includes street names, building numbers, apartment or suite numbers, and other address details.
Order			

OrdId	String	20	Order Identifier
CusId	String	50	Customer Identifier
ShippingAddress	String	255	Shipping address
OrDate	String	20	The date and time when the order was placed or created
Status	String	50	The current state or progress of the order through its lifecycle.
Note	String	50	The note provided by the customer when placing an order
TotalAmount	Int		The total cost of the order, including all items, taxes, discounts, and any other charges that may apply
PaymentMethod	String	30	The method or mode used by the customer to pay for the order
ShippingCost	Int		The total cost associated with shipping and delivering the order to the customer.
PaymentStatus	String	20	The current state of the payment for the order
OrderProduct			The OrderProduct entity is created from the many-to-many relationship between Order and Product.
ProId	String	20	Product identifier
Quantity	Int		Quantity of products that customers order
Price	int		Price of a product within each order
Discount	Float		discount of a product within each order
ImportBill			Import Bill

ImpId	String	20	Import bill Identifier
ImpDate	String	20	The date when the import of products into the store occurs
SupId	String	20	Supplier Identifier
StaffId	String	20	Identifier of the staff who processed or handled the import bill.
TotalAmount	int		The total monetary value of the import transaction
ImportBillProduct			The OrderProduct entity is created from the many-to-many relationship between ImportBill and Product.
ProId	String	20	Product Identifier

ImBId	String	20	Import Bill Identifier
ImportPrice	int		The price of each product when the store imports it
SellingPrice	int		The price at which the store intends to sell each product after it has been imported.
Quantity	Int		The quantity of each product when the store imports it
RemainingQuantity	int		The number of units of a specific product that are still available in the store's inventory from a particular import batch
ExpirationDate	String	50	The date after which a specific product is no longer considered safe or suitable for sale or consumption
staff			It represents employees within the store, who can play different roles such as sales staff, warehouse staff, and store owner.
StaId	String	20	Staff identifier
StaName	String	50	Staff Name
Role	String	20	Role of staff in the system
Phone	String	15	Phone number of staff
HireDate	DateTime		The date when an employee is officially hired or begins working at the organization.

User			Users of the system
Uid	String	20	User identifier
ExternalUserID	String	50	Unique identifier for users from an external system or service that is integrated with your system. This could be used to track users who log in through third-party authentication services like Google, Facebook, or other external identity providers.
ExternalProvider	String		The name of the external authentication service or provider that is used to authenticate or identify the user.
Password	String	50	The password of the user if they have
UserType	String	50	It categorizes users based on their roles or access levels within the system.

News			It typically represents articles that are communicated to customers.
NewId	String	20	New identifier
Title	String	255	Title of news
Content	String	max	Content of news
CreateDate	String	20	Time creates the news
StaffId	String	20	Staff identifier of who writes news
Log			Log is an entity used to record events, actions, or changes within a system
TimeStamp	DateTime		The date and time when the log entry was created.
StaId	String	20	The identifier of the user who performed the action
EntityID	String	20	The identifier of the entity related to the action (e.g., order ID, product ID). This attribute specifies which particular instance of an entity was involved.
EntityType	String	50	The type of entity associated with the action (e.g., order, product). It indicates what kind of data was affected.

Action	String	20	The type of action or event recorded (e.g., create, update, delete). It describes the nature of the activity.
Description	String	255	A detailed description of the event or action
Cart			
CarId	String	20	A unique identifier for each cart
CreateDate	DateTime		The date and time when the cart was created
Status	String		The status of the cart (e.g., "active," "abandoned," "completed")
CartItem			It represents an individual item within a customer's shopping cart. It captures details about each product the customer intends to purchase
ProId	String		A unique identifier for each product in the cart.
Quantity	Int		The number of units of the product added to the cart.
DateAdded	Date		The date and time when the item was added to the cart
Price	int		The price per unit of the product at the time it was added to the cart

II. Analysis models

II.1 Interaction diagrams

In this section, we will analyze objects and their interactions to realize use cases based on the MVC model's analysis pattern. Specifically, we will examine the following use cases: Manage Products, Add Product, Edit Product, Delete Product, Disable News, and Enable News. The interaction diagrams, illustrated in Figures 2-1 through 2-12 include sequence diagrams and communication diagrams that model the interactions (messages) of objects to realize these use cases.

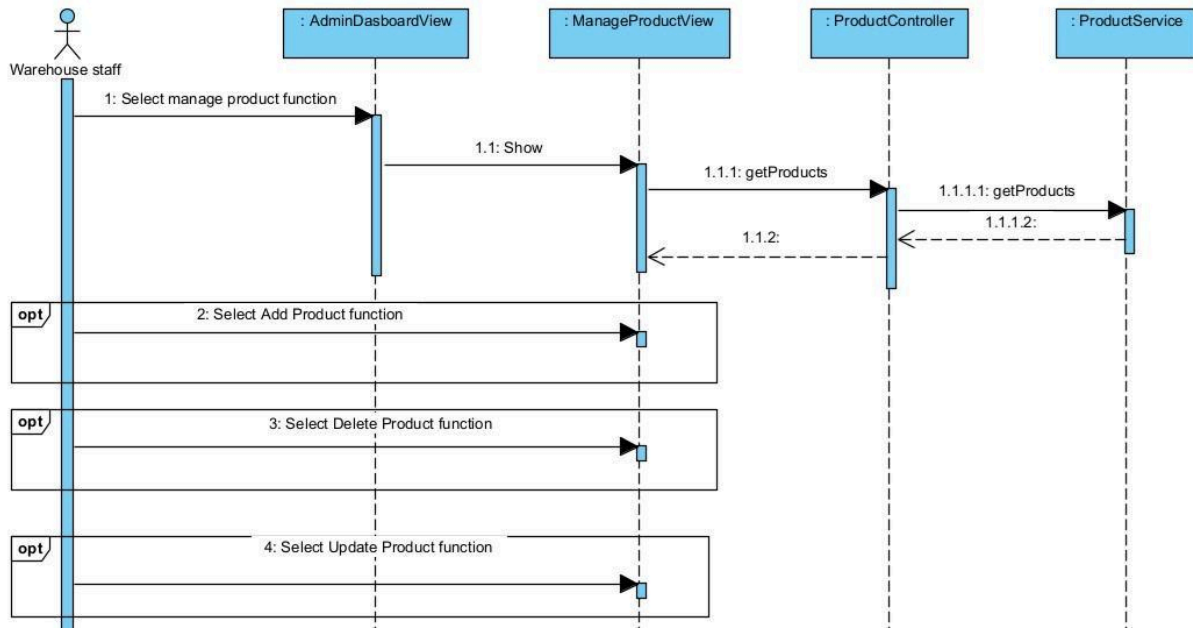


Figure II-1 Sequence Diagram for Manage Products Use Case

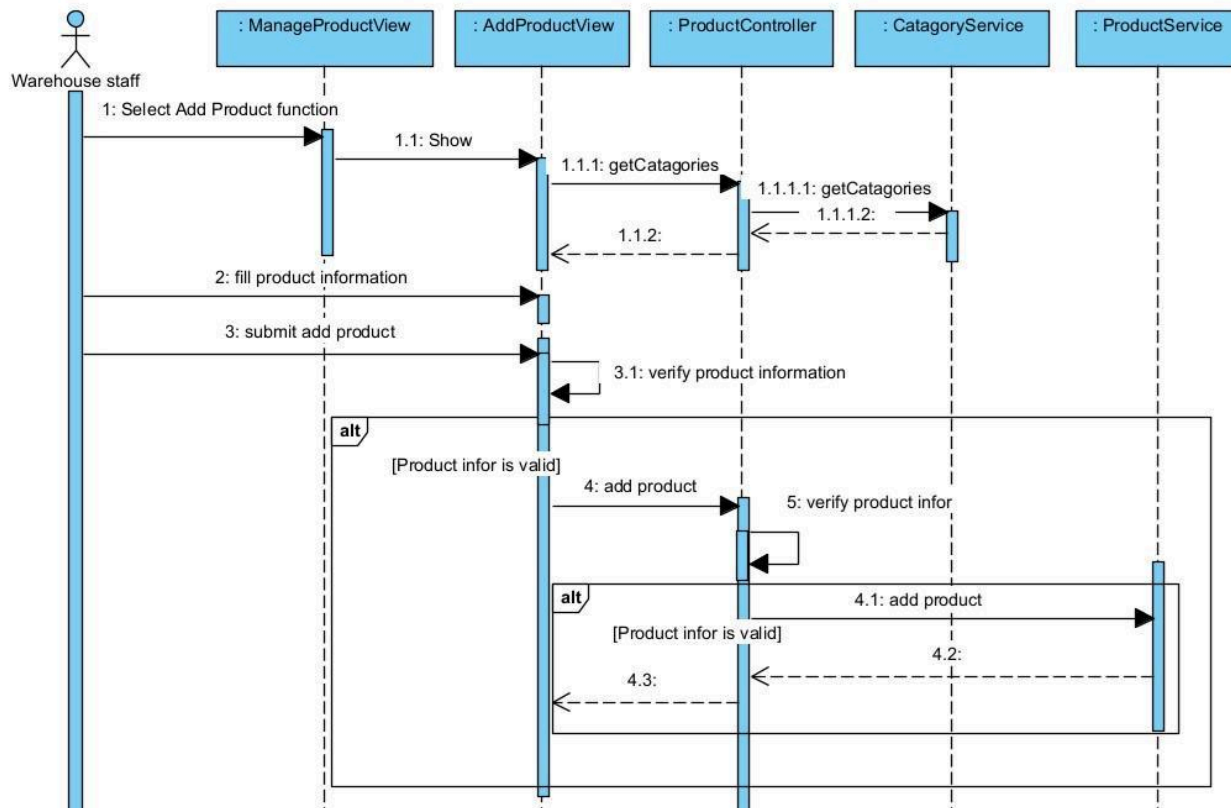


Figure II-2 Sequence Diagram for Add Product Use Case

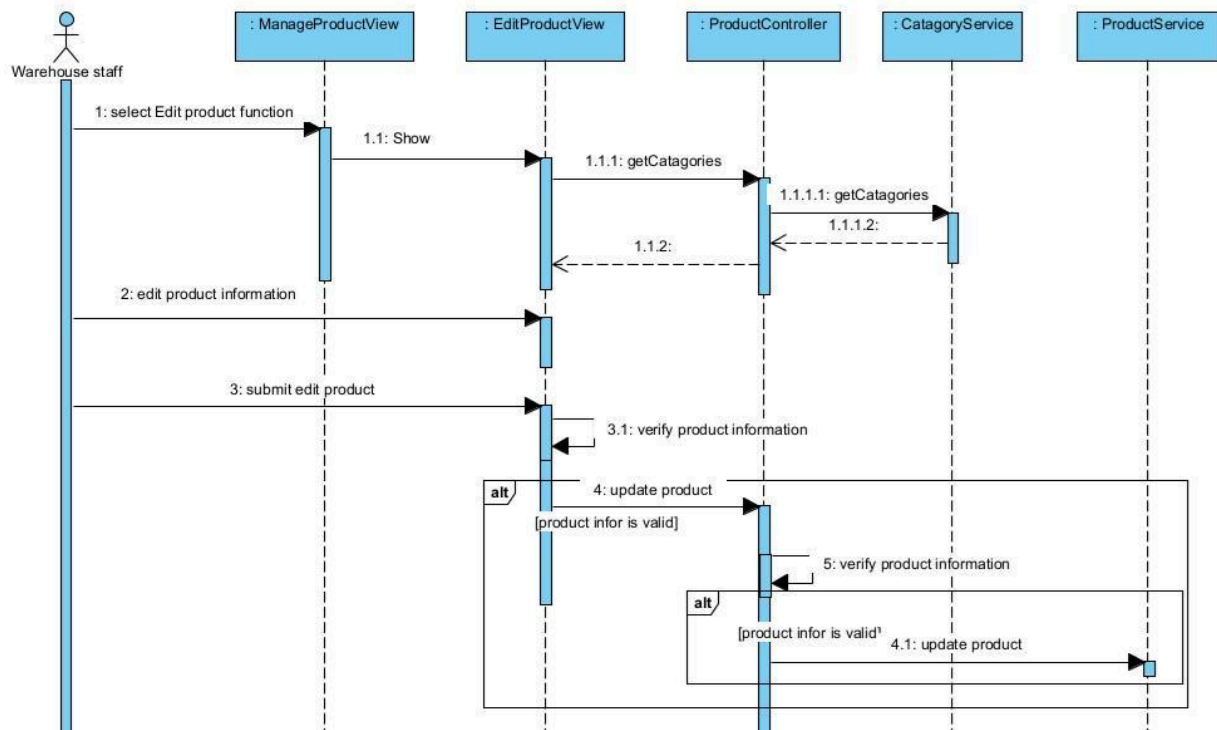


Figure II-3 Sequence Diagram for Edit Product Use Case

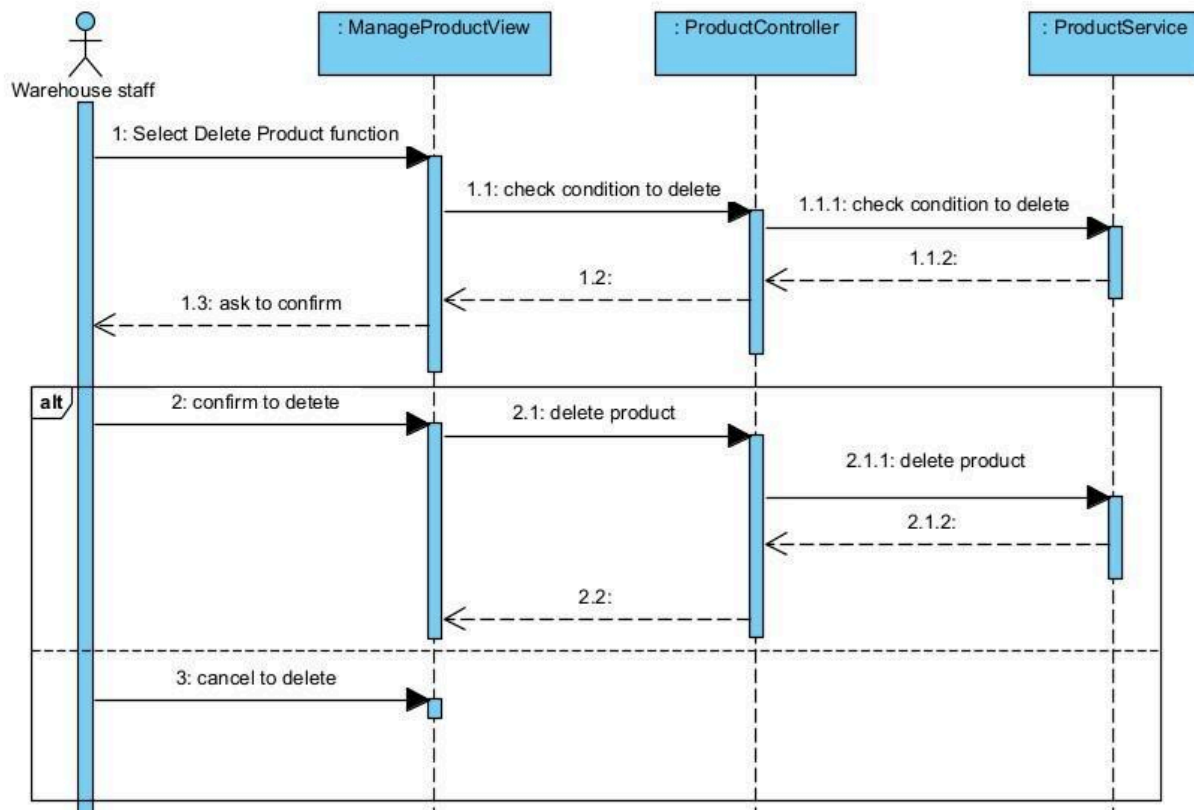


Figure II-4 Sequence Diagram for Delete Product Use Case

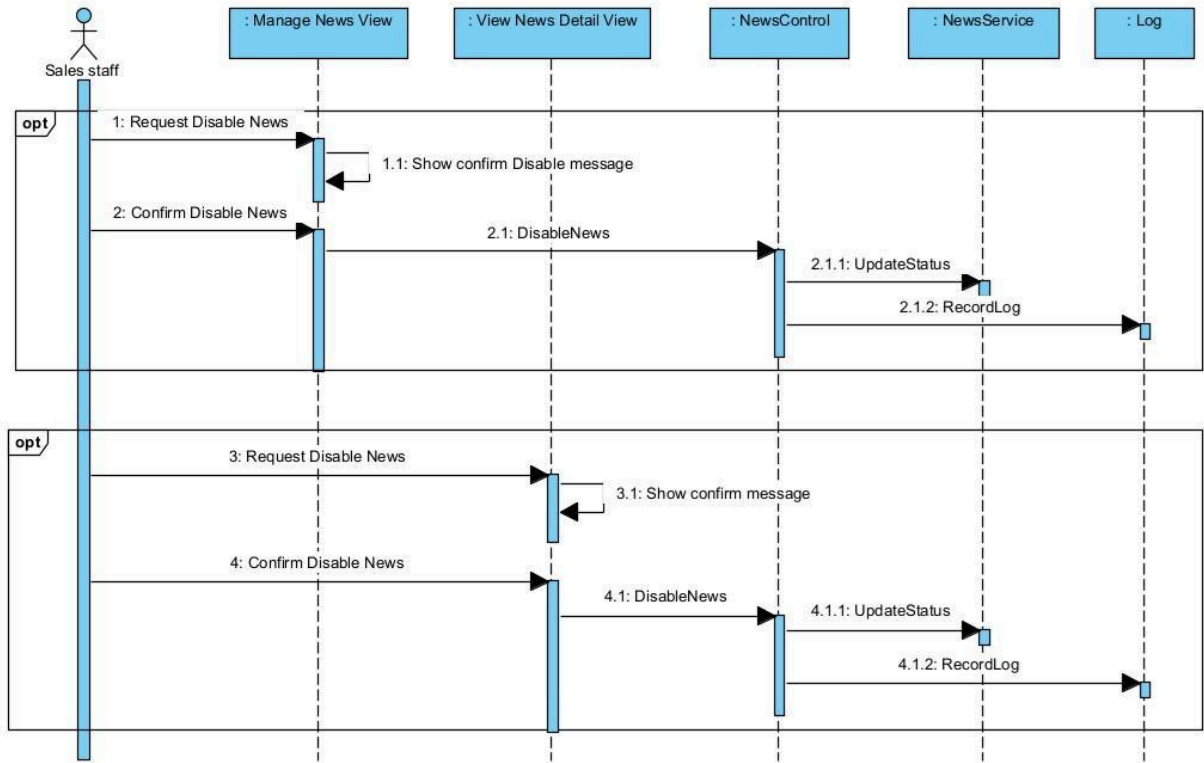


Figure II-5 Sequence Diagram for Disable News Use Case

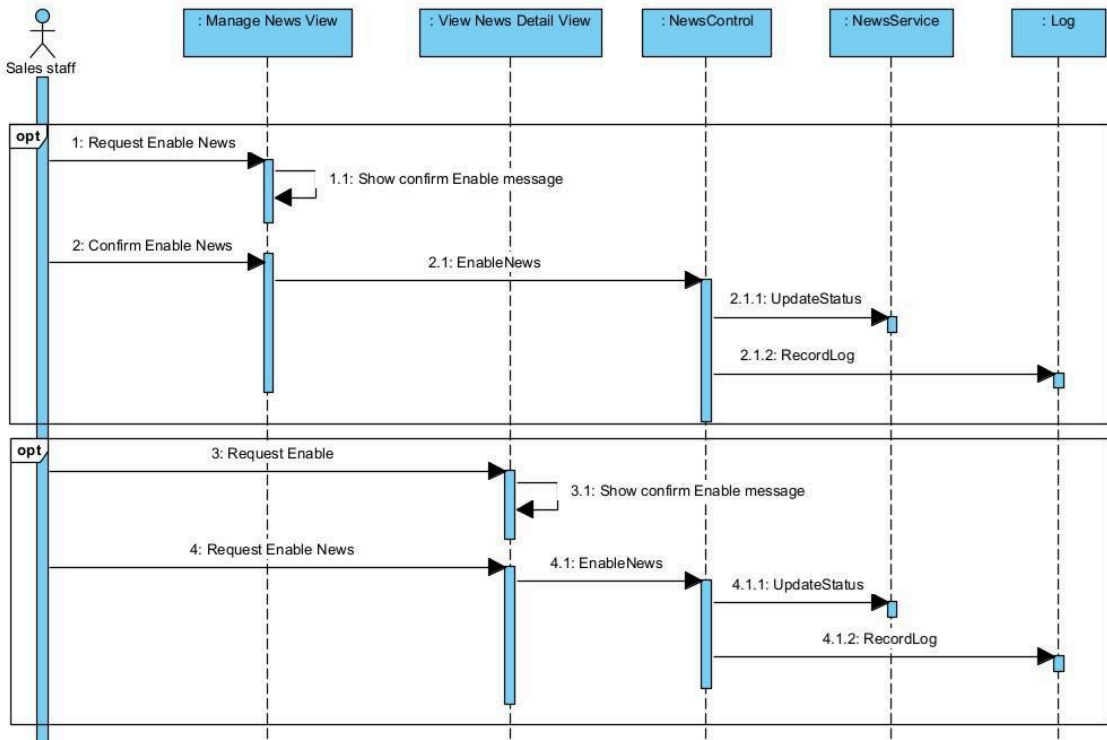


Figure II-6 Sequence Diagram for Enable News Use Case

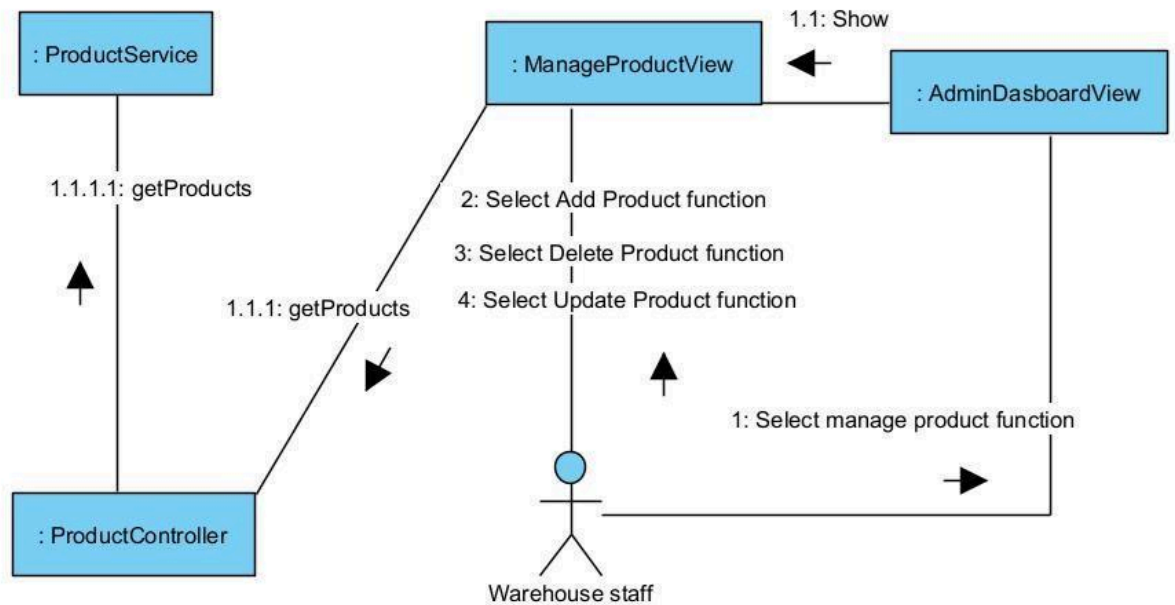


Figure II-7 Communication Diagram for Manage Products Use Case

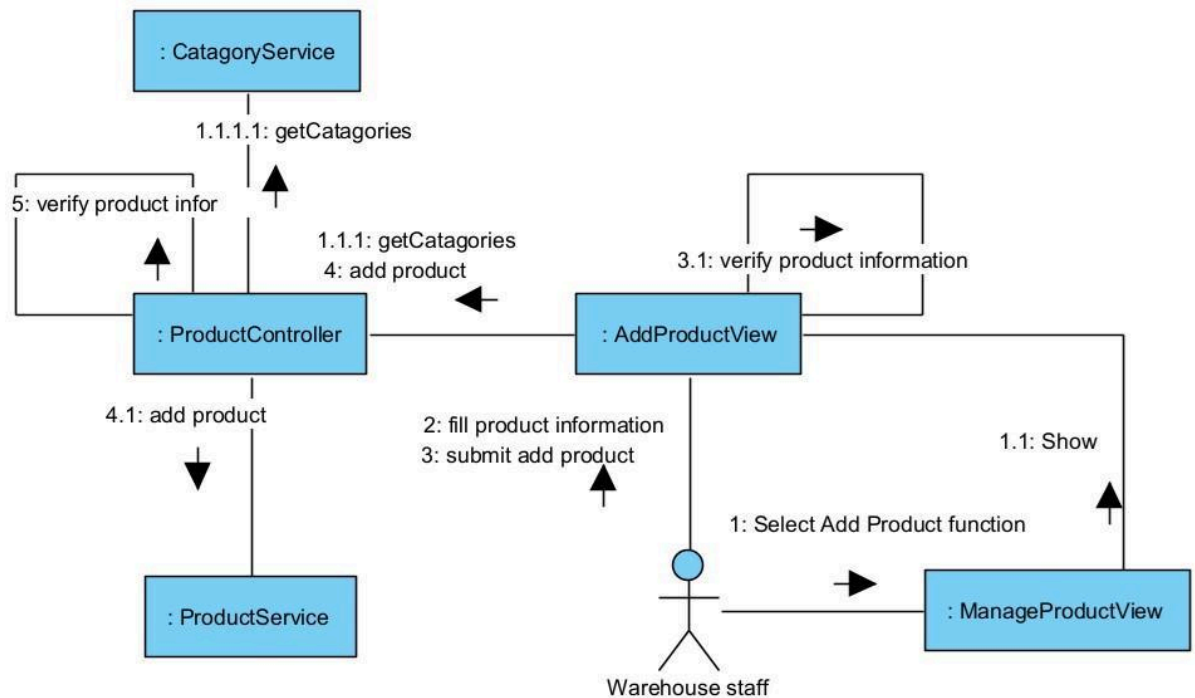


Figure II-8 Communication Diagram for Add Product Use Case

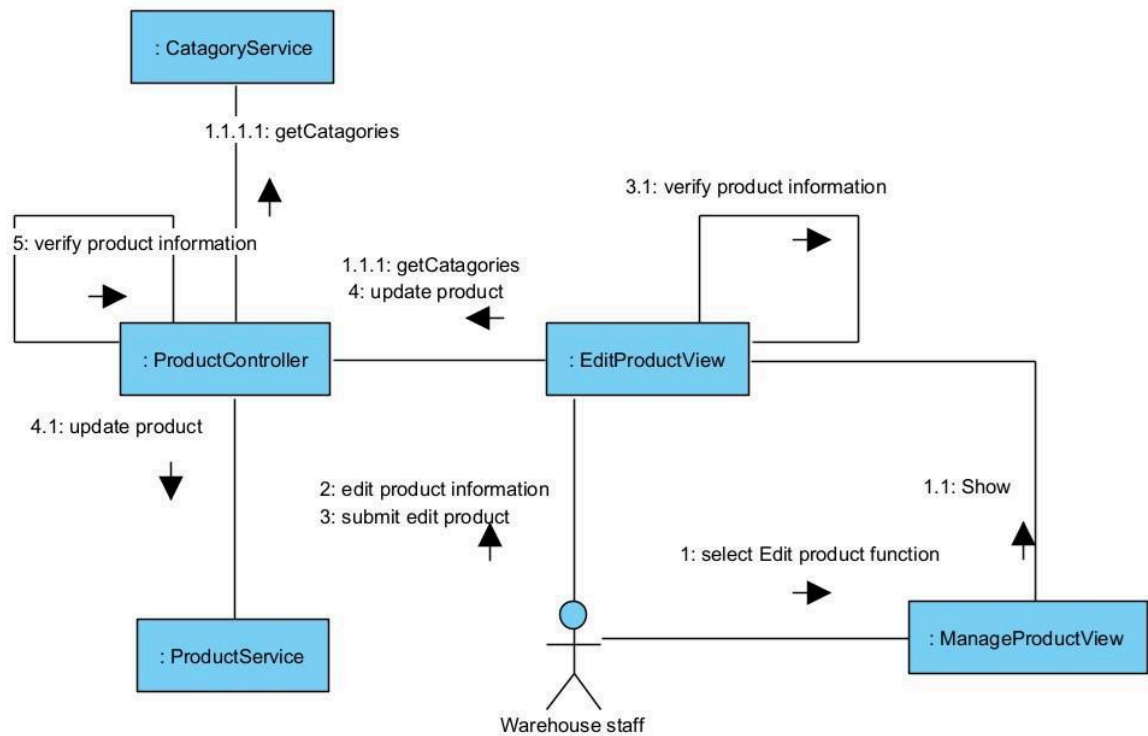


Figure II-9 Communication Diagram for Edit Product Use Case

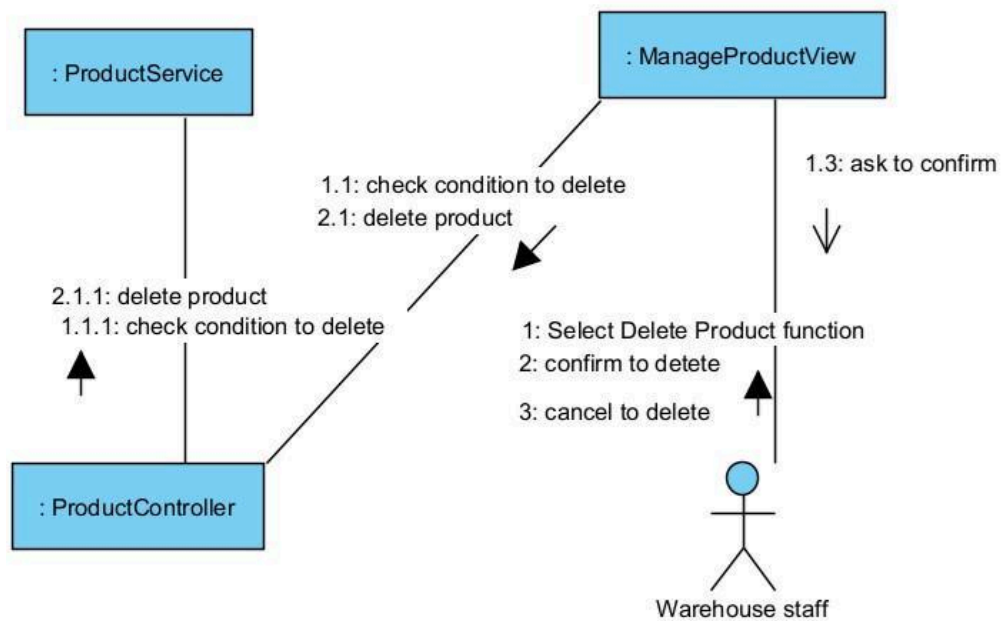


Figure II-10 Communication Diagram for Delete Product Use Case

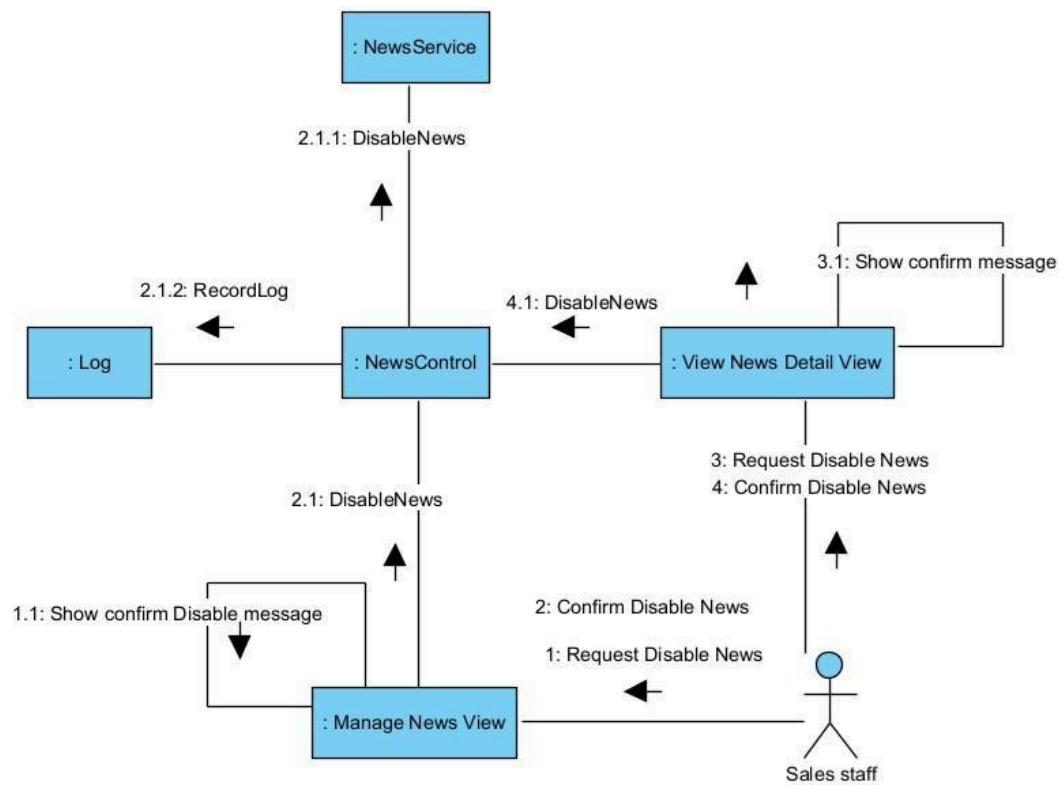


Figure II-11 Communication Diagram for Disable News Use Case

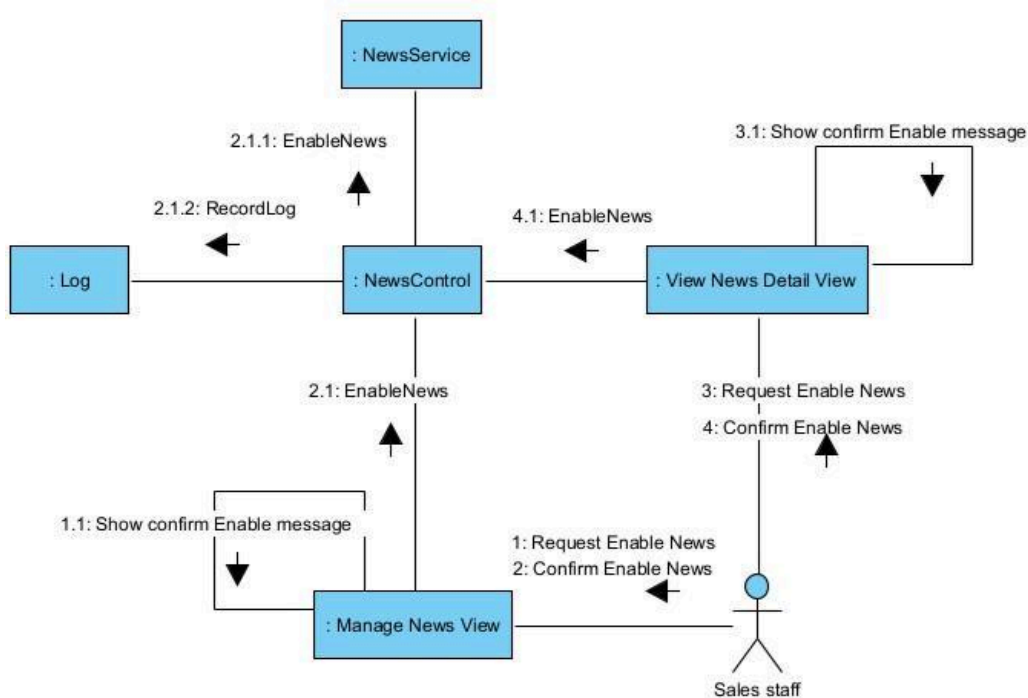


Figure II-12 Communication Diagram for EnableNews Use Case

II.2 State diagram

In object-oriented analysis and design, a finite state machine can model the states and state transitions of a system or an object. Figure 2-13 illustrates a state diagram depicting the states and state transitions of a News object. The events DisableNews and EnableNews cause state transitions to share the same names as two messages arriving at the state-dependence object in the sequence diagrams (Figures 2-5, 2-6) or the communication diagrams (Figures 2-11, 2-12).

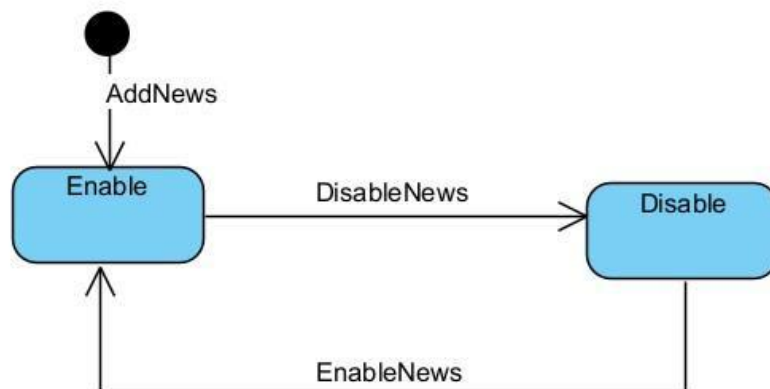


Figure II-13 State diagram model states of a News object

III. Design specification

III.1 Integrated Communication Diagrams

To transition from analysis to design and to identify the subsystems, it is necessary to synthesize an initial software design based on the analysis conducted so far. This involves integrating the use case based interaction diagrams developed as part of the dynamic model [1].

The integrated collaboration diagrams will then incorporate all interactions within the system. For large and complex systems, this integration may be performed for each subsystem individually. In this example, we will integrate the collaboration diagrams for the four use cases we have developed: Manage Products, Add Product, Edit Product, and Delete Product.

As an alternative to integrating interactive diagrams, design often involves incorporating class diagrams. This integration helps identify all the necessary classes and methods required to implement use cases. For example, Figure 3-7 shows a design-level class diagram that integrates the use cases: Manage Products, Add Product, Edit Product, and Delete Product.

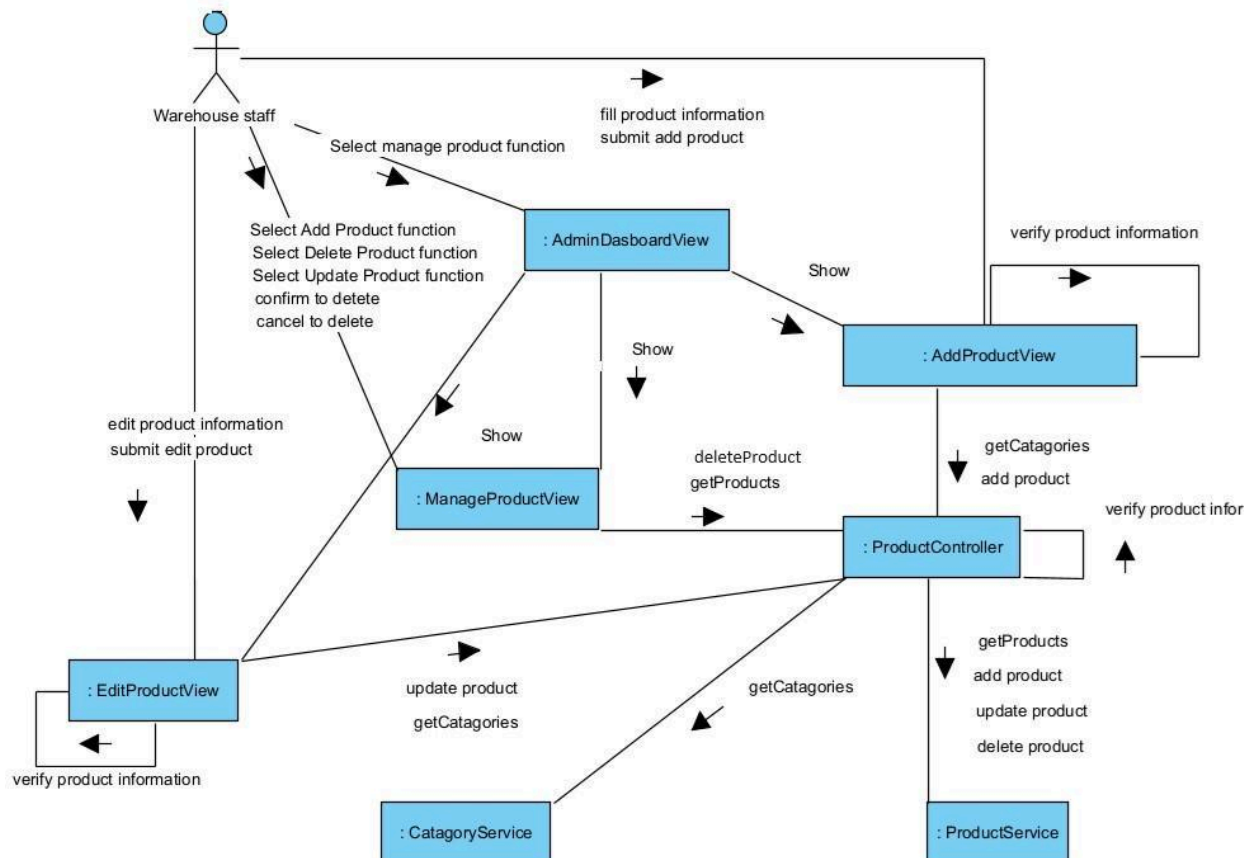


Figure III-1 Integrated communication diagram of Manage products, Add product, Edit product, Delete product.

III.2 System High-Level Design

The software architecture separates the overall structure of the system. The design of the software architecture can be depicted from different perspectives, referred to as different views. The static view of the software architecture is depicted on class diagrams. The dynamic view of the software architecture is depicted on communication diagrams. The deployment view of the software architecture is depicted deployment diagram. Another view of software architecture is the Component-Based view [1].

Based on the functional and non-functional requirements, the FSO system is designed using a Multiple Client/Single Service architecture. In this design, the FSO system is divided into two subsystems: one subsystem for customers and another subsystem for store staff. The software highlevel architecture of FSO is depicted by different views as in Figures 3-2, 3-3, and 3-4.

Note: To meet the non-functional requirements NF-02 and NF-03, we need to design the interface to ensure ease of use, enable the website to run on multiple browsers, and support Responsive capabilities. Additionally, to meet the non-functional requirements NF-01 regarding maintainability and NF-04 regarding performance, we need to design the architecture to fulfill these characteristics.

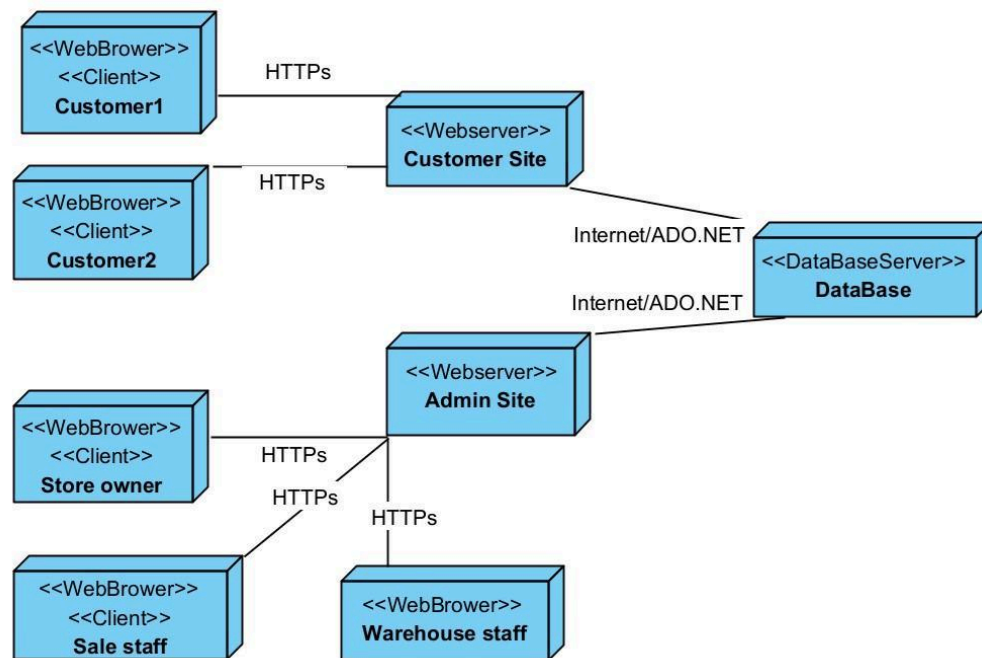


Figure III-2 Deployment diagram for the FSO system architecture

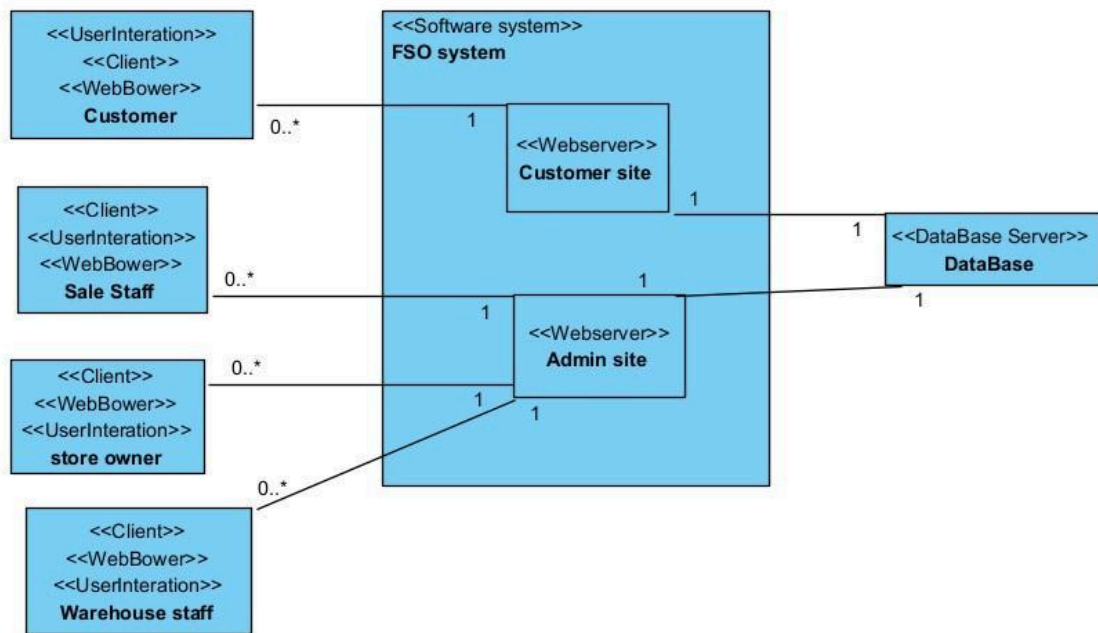


Figure III-3 Class diagram for the FSO system architecture

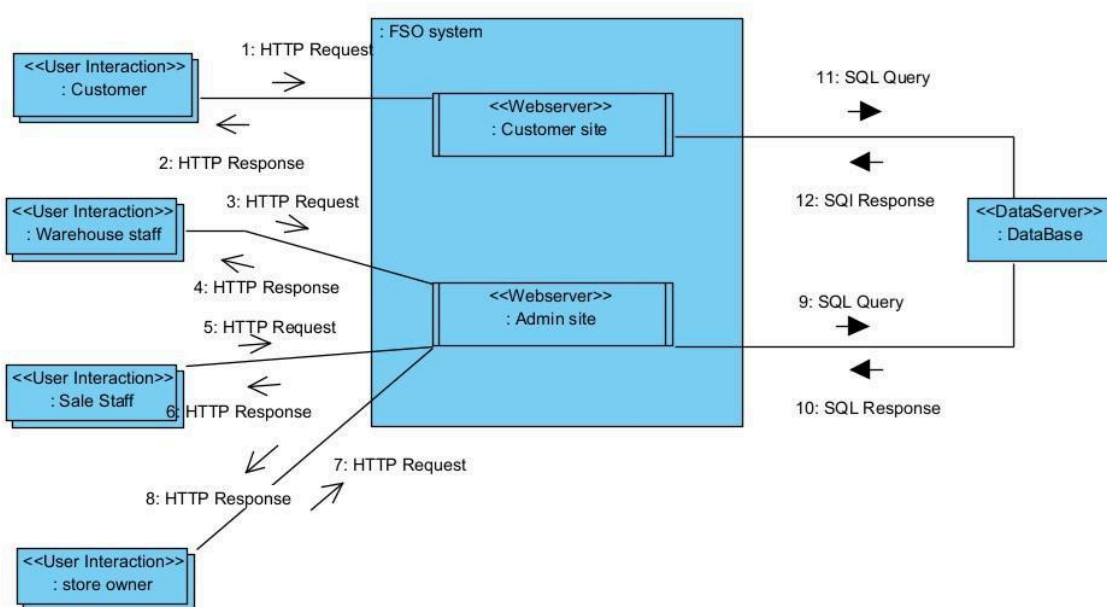


Figure III-4 Component diagram for the FSO system architecture

III.3 Component and Package Diagram

After designing the system into subsystems, the organization of classes within the subsystems into packages and components can also be determined. This organization is modeled in the corresponding package and component diagrams in UML. Figure 3-5 is the package diagram of the FSO system. This diagram shows packages and the relationships between packages. Figure 3-6 is a component diagram of the FSO system. This diagram shows components and the relationships between components.

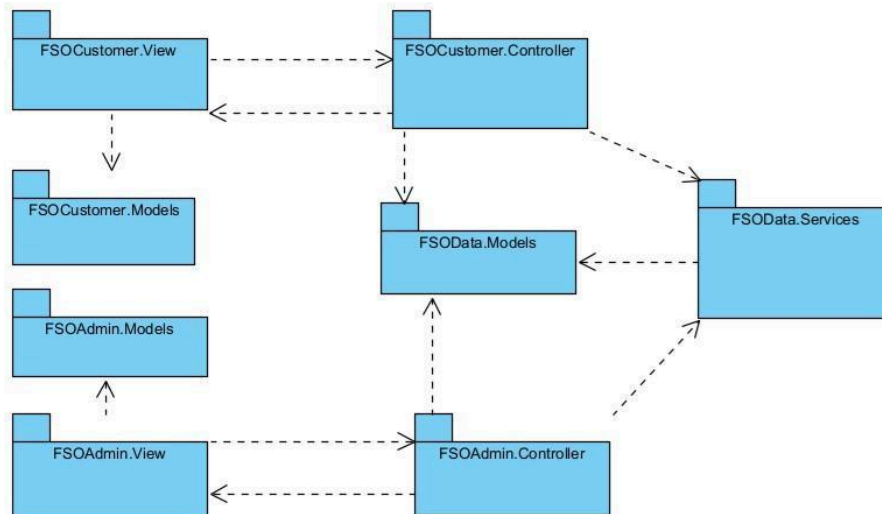


Figure III-5 Package diagram for the FSO system

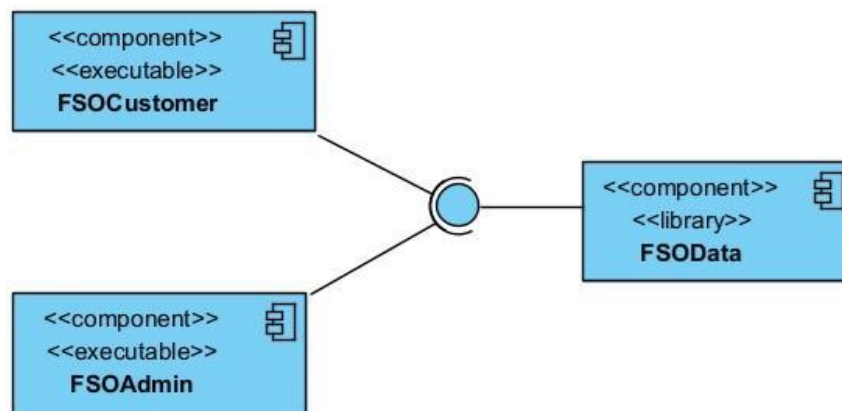


Figure III-6 Component diagram for the FSO system

III.4 Detail Design

After high-level architectural design, the classes in the analysis model are mapped to design classes. The operations of a class can be determined from either the static model and the dynamic model. Information hiding classes are categorized by stereotype: entity classes, boundary classes, control classes, and application logic classes. In design modeling, each entity

class is usually split into two separate classes. The first is a database wrapper class, which hides how data is accessed, especially when using a relational database. The second is a data abstraction class, which encapsulates the data structures [1].

Figure 3-7 shows the detailed design classes for the use cases: Manage Products, Add Product, Edit Product, and Delete Product. These classes are identified based on the architecture of the FSO system, as designed in Section 3.2. The operations of these classes are defined based on the messages in the sequence diagrams for these use cases, as shown in Figures 2-1 through 24.

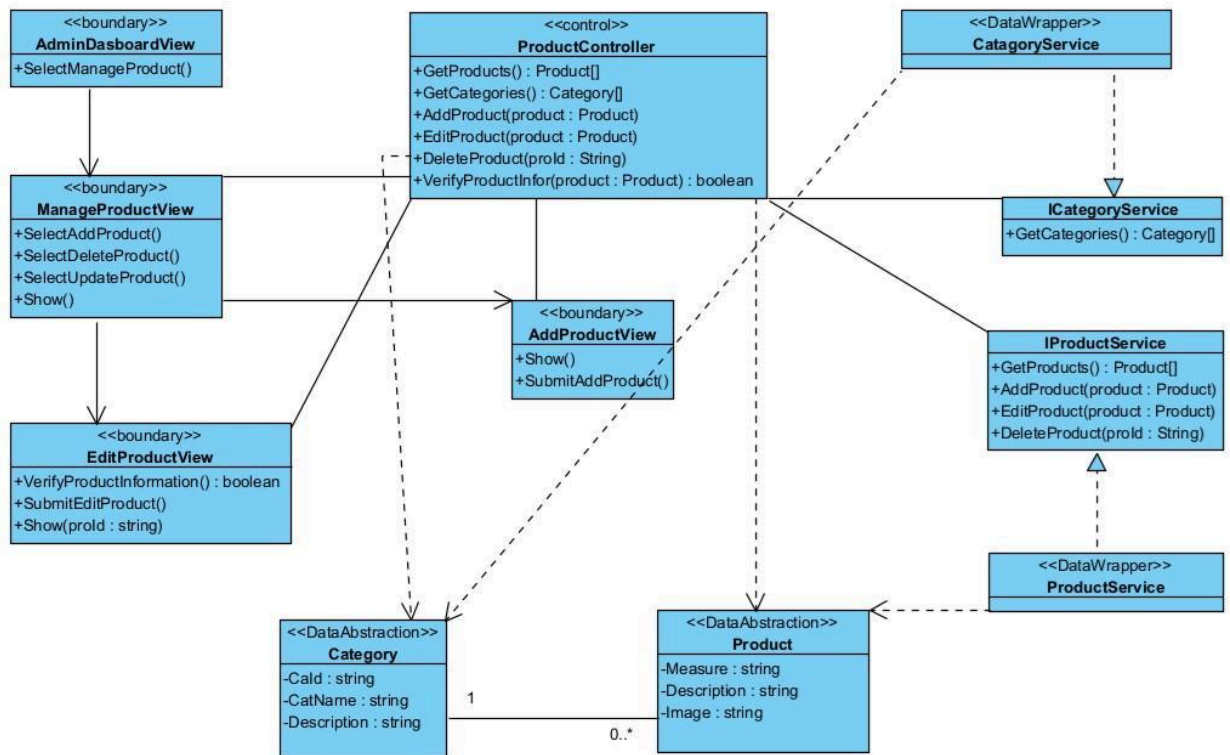


Figure III-7 Detailed Design Classes for Use Cases: Manage Products, Add Product, Edit Product, Delete Product

III.5 Database Design

The data contained in the entity classes of the static model are mapped to a database (in this system, we will design a relational database). The entity class diagram of the FSO system, shown in Figure 1-4, which models the data requirements, will be mapped to a relational database design for the system.

Supplier(SupId: nvarchar(20), Name: nvarchar(50), Address: nvarchar(500), Phone: nvarchar(15))

Customer(CusId: nvarchar(20), CusName:nvarchar(50), PhoneNumber:nvarchar(15), Email:nvarchar(50), Address: nvarchar(500))

Staff(StaId: nvarchar(20), StaName: nvarchar(50), Role: nvarchar(20), Phone: nvarchar(15), HireDate: DateTime)

Discount(DisId: nvarchar(20), DisName: nvarchar(250), DisAmount: float, IsPercentage, StartDate: DateTime, EndDate: DateTime, IsActive: bit)

Staff(StaId: nvarchar(20), StaName: nvarchar(50), Role: nvarchar(20), Phone: nvarchar(15), HireDate: DateTime)

User(Uid: nvarchar(20), *StaId:nvarchar(20)*, *CusId: nvarchar(20)*, ExternalUserID:nvarchar(50), ExternalProvider:nvarchar(50), Password:nvarchar(50), UserType: nvarchar(50))

News(NewId: nvarchar(20), *StaffId: nvarchar(20)*, Title: nvarchar(255), Content: LongText, CreateDate: DateTime)

Log(LogId: nvarchar(20), TimeStamp: DateTime, *StaId: nvarchar(20)*, EntityID: nvarchar(20), EntityType: nvarchar(50), Action: nvarchar(20), Description: nvarchar(255))

Category(CatId: nvarchar(20), CatName: nvarchar(50),Description: nvarchar(500))

Product(ProId: nvarchar(20), *CatId: nvarchar(20)*, ProName: nvarchar(50), Measure: nvarchar(50), Description: nvarchar(500), Image: nvarchar(255))

Cart(CusId: nvarchar(20), CreateDate: DateTime, Status: nvarchar(20))

ShippingAddress(AddId: nvarchar(20), *CuId: nvarchar(20)*, Province: nvarchar(50), District: nvarchar(50), Commune: nvarchar(50), AddressLine: nvarchar(100))

Order(OrdId: nvarchar(20), *CusId: nvarchar(20)*, *StaId: nvarchar(20)*, ShippingAddress: nvarchar(255), OrDate, Status, Note, TotalAmount, PaymentMethod, ShippingCost, PaymentStatus)

ImportBill(ImpId: nvarchar(20), *SupId: nvarchar(20)*, *StaffId: nvarchar(20)*, ImpDate: DateTime, TotalAmount: int)

CartItem(CaId: nvarchar(20), *CusId: nvarchar(20)*, ProId: nvarchar(20), Quantity: int, DateAdded: DateTime, Price: int)

ImportBillProduct(ProId: nvarchar(20), ImBId: nvarchar(20), ImportPrice: int, SellingPrice:int, Quantity: int, RemainingQuantity: int, ManufactureDate: DateTime, ExpirationDate: DateTime)

OrderProduct(OrdId: nvarchar(20), ProId: nvarchar(20), Quantity: int, Price: int, Discount:Float)

IV. Implementation

IV.1 Map architecture to the structure of the project

In the system architecture outlined in Section 3.2, the FSO (FreshFood Online) system is designed using a Multi-client/one-server model. It is divided into two subsystems: FSOAdmin and FSOCustomer, deployed in a web environment. To manage data operations effectively, these tasks are separated into a distinct component. As illustrated in Figure 3-6 (Component Diagram), the system consists of three main components: FSOAdmin (Executable - Web app), FSOCustomer (Executable - Web app), and FSOData (Library). These components are further organized into packages, as shown in Figure 3-5. Each package groups classes with similar roles and functionalities.

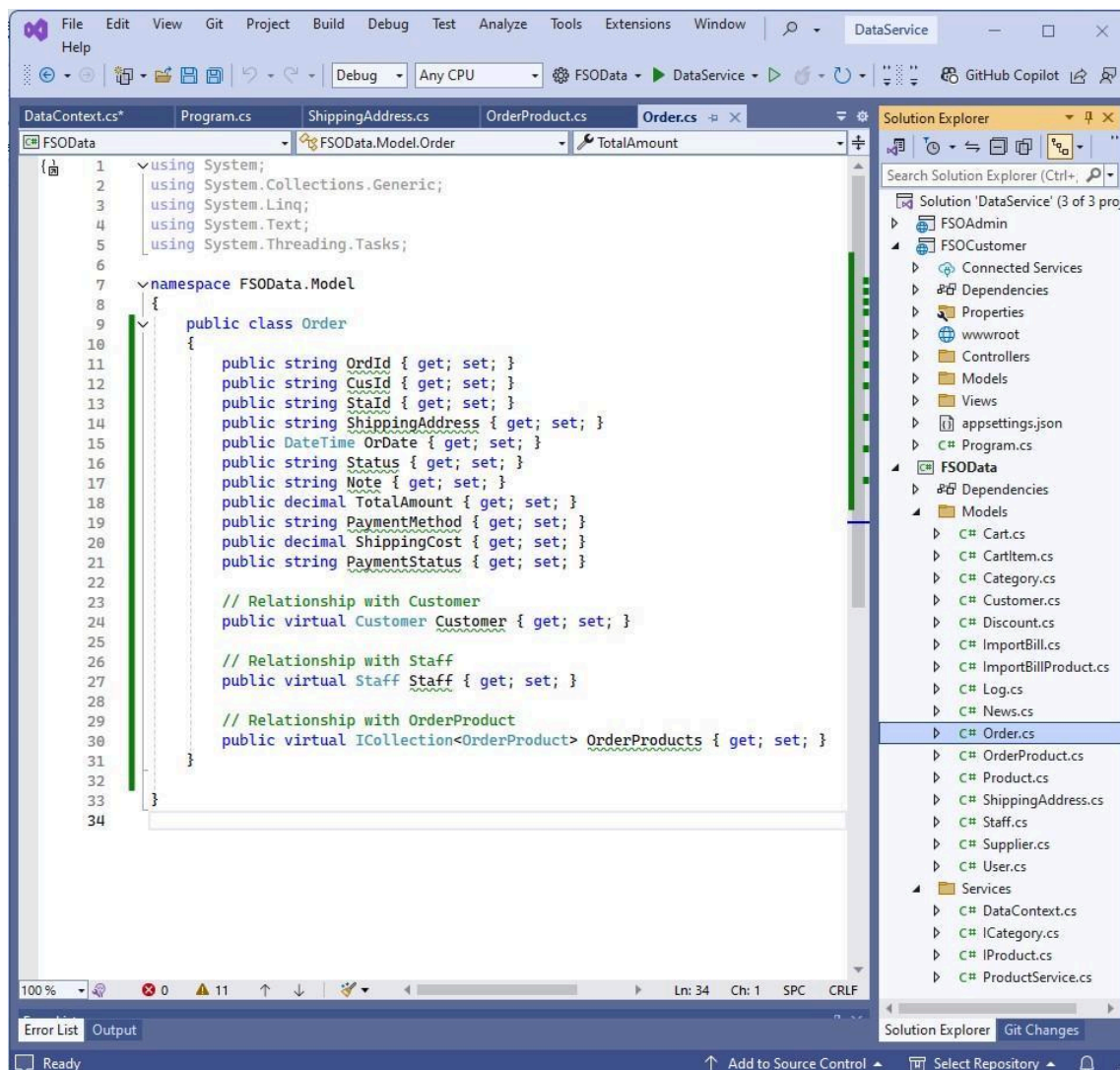


Figure IV-1 the structure of the project

IV.2 Map Class Diagram and Interaction Diagram to Code

For example, two classes (ProductService, ProductController) in the detailed design classes in Figure 3-7 are mapped into code c# as follows:

```
using          FSOData.Models;
namespace FSOData.Services
{ public interface IProductService
    {
        Task<IEnumerable<Product>> GetProducts ();
        Task AddProduct (Product product);
        Task EditProduct(Product product);
        Task DeleteProduct(string proId);
    }
}

using          FSOData.Models;
namespace FSOData.Services
{
public class ProductService : IProductService
{
    private readonly DataContext _context;

    public ProductService(DataContext context)
    {
        _context = context;
    }

    // Method to get all products
    public async Task<IEnumerable<Product>> GetProducts ()
```

```

    {
        return await _context.Products.ToListAsync();
    }

    // Method to add a new product    public async
Task AddProduct (Product product)
    {
        _context.Products.Add(product);
await _context.SaveChangesAsync();
    }

    // Method to update an existing product    public
async Task EditProductAsync(Product product)
    {
        _context.Products.Update(product);
await _context.SaveChangesAsync();
    }

    // Method to delete a product by ID
public async Task<bool> DeleteProduct (string proId)
    {
        // Check if the product exists in any related tables (e.g., OrderProduct,
ImportBillProduct)    bool isReferenced = await _context.OrderProducts.AnyAsync(op =>
op.ProId == proId) ||                await _context.ImportBillProducts.AnyAsync(ibp =>
ibp.ProId == proId);    if (isReferenced) return false;    var product = await
_context.Products.FindAsync(proId);    if (product != null)
    {
        _context.Products.Remove(product);
    }
}

```

```

        await _context.SaveChangesAsync();
return true; // Deletion successful
    }
    return false; // Product not found
}
}
}

using FSOData.Models; using
FSOData.Services; namespace
FSOAdmin.Controller
{ public class ProductController:
Controller
{
    private readonly IProductService
    _productService; private readonly
    ICategoryService _categoryService;

    // Constructor public
    ProductController(IProductService productService)
    {
        _productService = productService;
    }

    // GET: Product/Manage public async
    Task<IActionResult> Manage()
    {
        var products = await _productService.GetProducts();
return View(products);

```

```

    }

    // GET: Product/GetProduct/{id}    public async
Task<IActionResult> GetProduct(string id)
{
    var product = await _productService.GetProduct(id);
    if (product == null)
    {
        return NotFound();
    }
    return PartialView("_ProductDetails", product);
}

    // GET: Product/GetCategories    public async
Task<IActionResult> GetCategories()
{
    var categories = await _categoryService.GetCategories();
    return Json(categories);
}

    // GET: Product/Add
public IActionResult Add()
{
    return PartialView("_AddProductModal");
}

    // POST: Product/Add
[HttpPost]

```

```

[ValidateAntiForgeryToken] public async
Task<IActionResult> Add(Product product) { if
(ModelState.IsValid)
{
    bool isVerified = _productService.VerifyProductInfor(product);
    if (!isVerified)
    {
        ModelState.AddModelError("", "Product information is not valid.");
        return PartialView("_AddProductModal", product);
    }

    await _productService.AddProduct(product);
    return RedirectToAction("Manage");
}

    return PartialView("_AddProductModal", product);
}

// GET: Product/Edit/{id} public async
Task<IActionResult> Edit(string id)
{
    var product = await _productService.GetProduct(id);
    if (product == null)
    {
        return NotFound();
    }

    return PartialView("_EditProductModal", product);
}

```

```

    }

    // POST: Product/Edit
    [HttpPost]
    [ValidateAntiForgeryToken] public async
Task<IActionResult> Edit(Product product)
    {
        if
(ModelState.IsValid)
        {
            bool isVerified = _productService.VerifyProductInfor(product);
            if (!isVerified)
            {
                ModelState.AddModelError("", "Product information is not valid.");
                return PartialView("_EditProductModal", product);
            }

            await _productService.EditProduct(product);
            return RedirectToAction("Manage");
        }

        return PartialView("_EditProductModal", product);
    }

    // POST: Product/Delete/{id}
    [HttpPost]
    [ValidateAntiForgeryToken] public async
Task<IActionResult> Delete(string id)
    {

```

```

        var product = await _productService.GetProduct(id);
    if (product == null)
    {
        return NotFound();
    }

    bool isDeleted = await _productService.DeleteProduct(id);
    if (isDeleted)
    {
        return Json(new { success = true, message = "Product deleted successfully." });
    }

    return Json(new { success = false, message = "Failed to delete the product. It may be
referenced in other records." });
}
}
}

```

V. Applying Alternative Architecture Patterns

Assuming that the functional requirements remain unchanged, the system is now being expanded to incorporate additional characteristics related to scalability and reusability. This enhancement allows the system to support the integration of new user-facing applications, including the potential development of a mobile app.

V.1 Applying the service-oriented architecture

NF-05 Reusability: The system should be built in a way that allows us to easily use the same code and features across different platforms. This means that the same functionality can be adapted for both the web and mobile apps, saving time and effort when adding new platforms or updating existing ones.

The current system architecture described in Section 3.2 does not fully meet the requirements of NF-05. To address this, the FSOData component must be restructured as a distributed component and designed using a service-oriented architecture (SOA). This adjustment will allow FSOData to function as a distributed service, thereby enhancing the system's ability to achieve Reusability, traceability, modularity, and effective maintenance. The software architecture is redesigned as shown below.

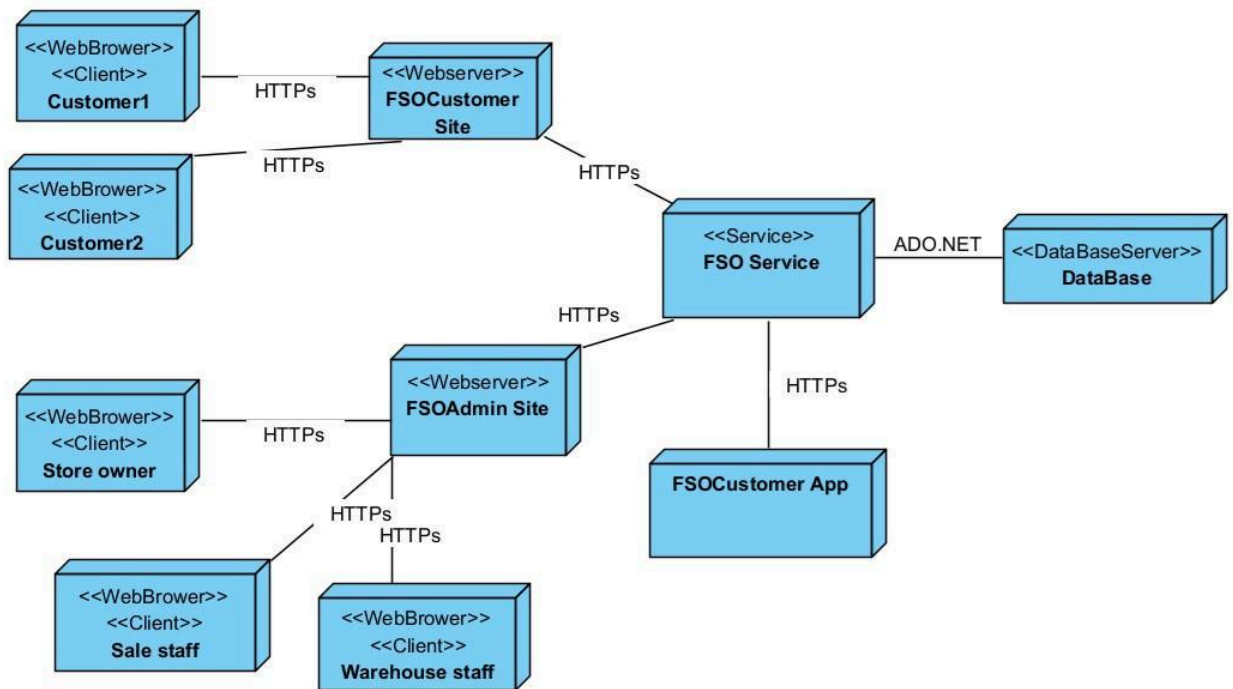


Figure V-1 Deployment Diagram for the FSO System Architecture Based on Service-Oriented Architecture

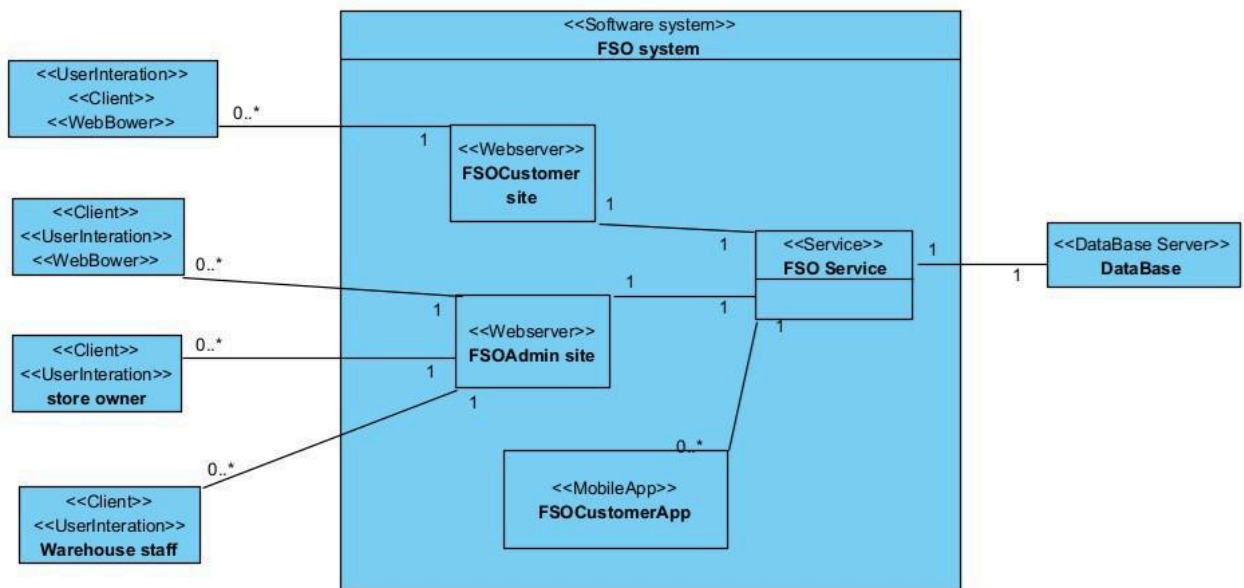


Figure V-2 Class Diagram for the FSO System Architecture Based on Service-Oriented Architecture

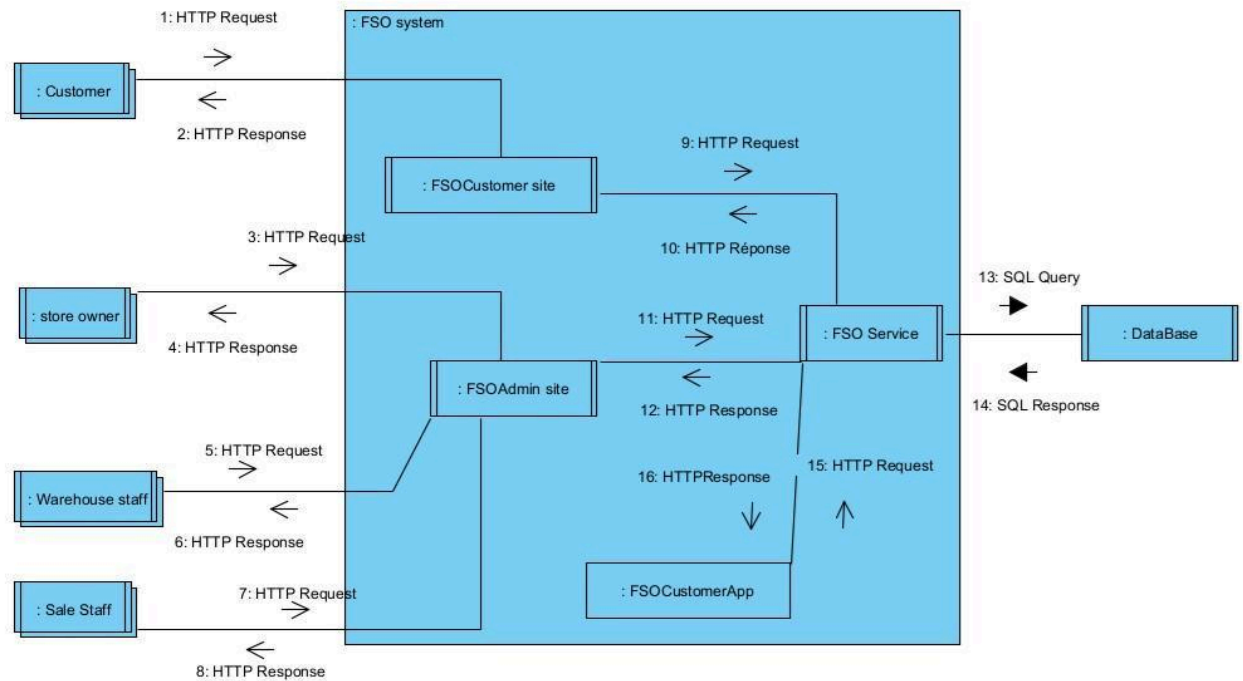


Figure V-3 Component Diagram for the FSO System Architecture Based on ServiceOriented Architecture

V.2 Applying Service Discovery Pattern in the service-oriented architecture

Assuming that user requirements change and the store's scale may later expand into a chain of stores with an increasing number of users, the system will need to address an additional nonfunctional requirement, NF-06, as follows:

NF-06: The system must be designed to accommodate an increasing number of stores and users while maintaining performance and reliability. This includes the ability to scale both horizontally and vertically. The architecture should support the addition of new stores and handle a growing volume of transactions and concurrent users without significant performance degradation. The system should implement load balancing to evenly distribute traffic and employ scalable database solutions to manage increasing amounts of data effectively.

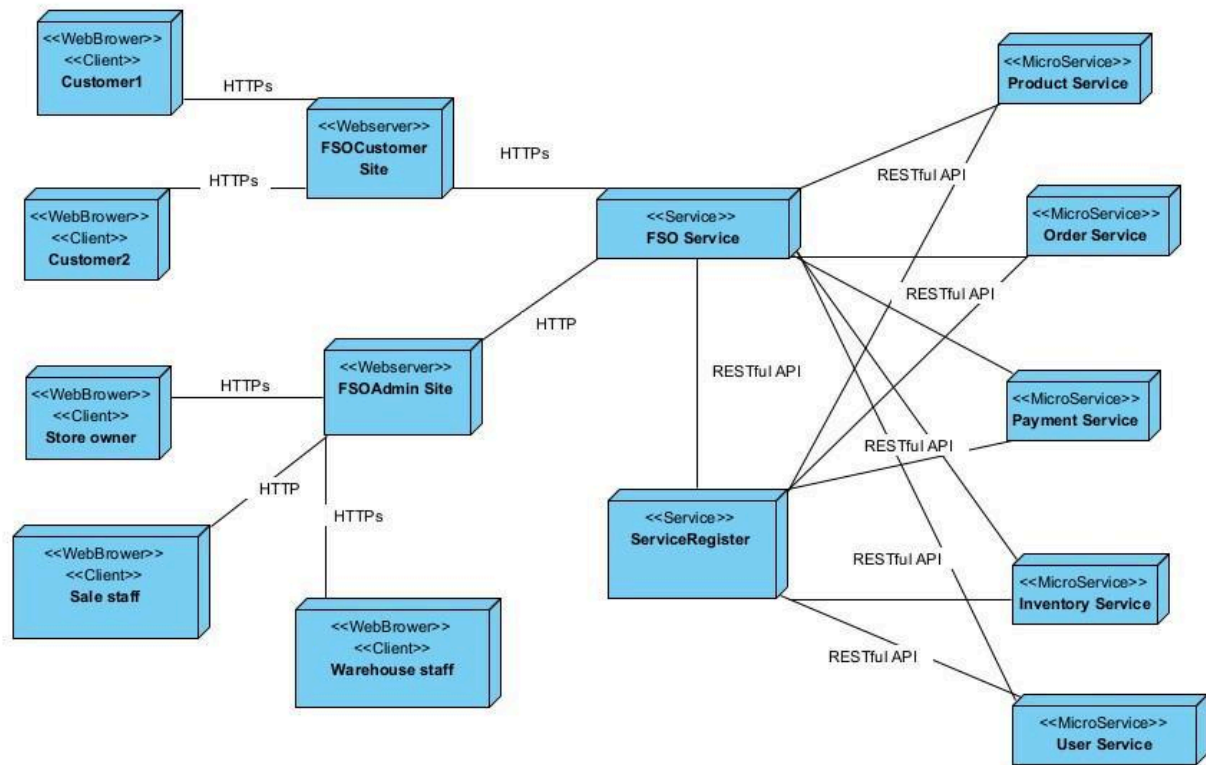


Figure V-4 Deployment Diagram for the FSO System Architecture Based on Microservice Architecture

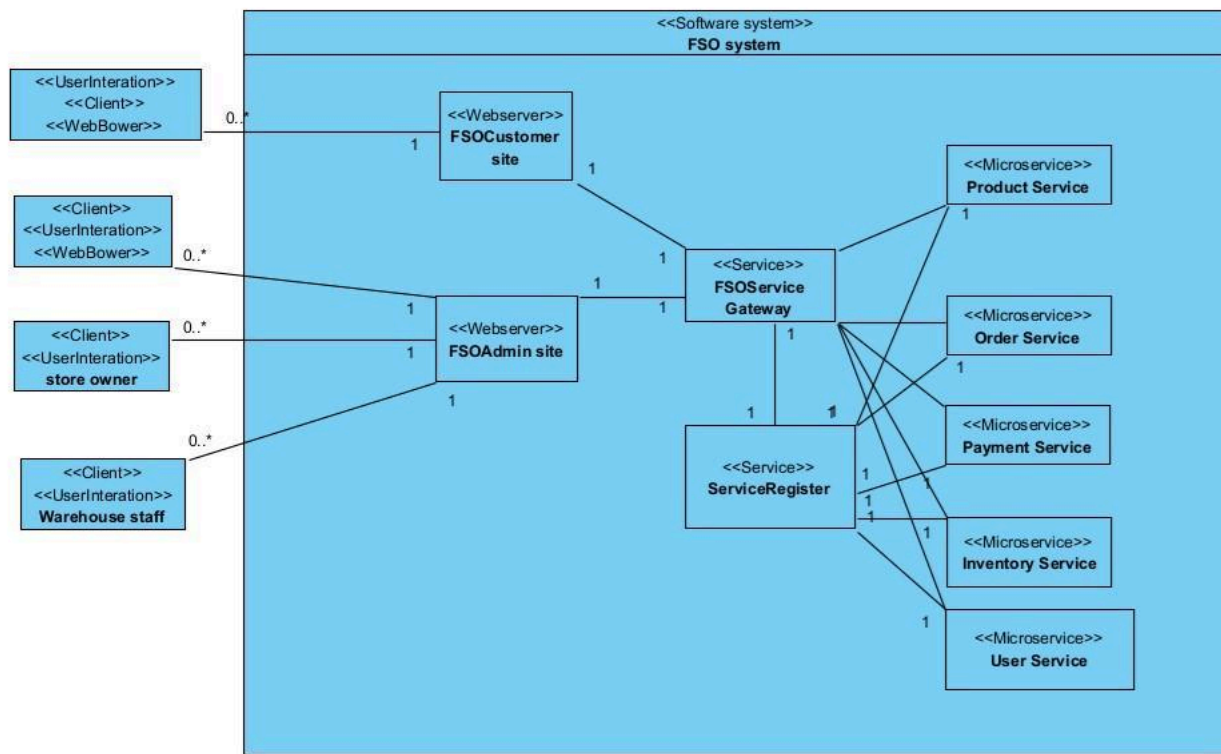


Figure V-5 Class Diagram for the FSO System Architecture Based on Microservice Architecture

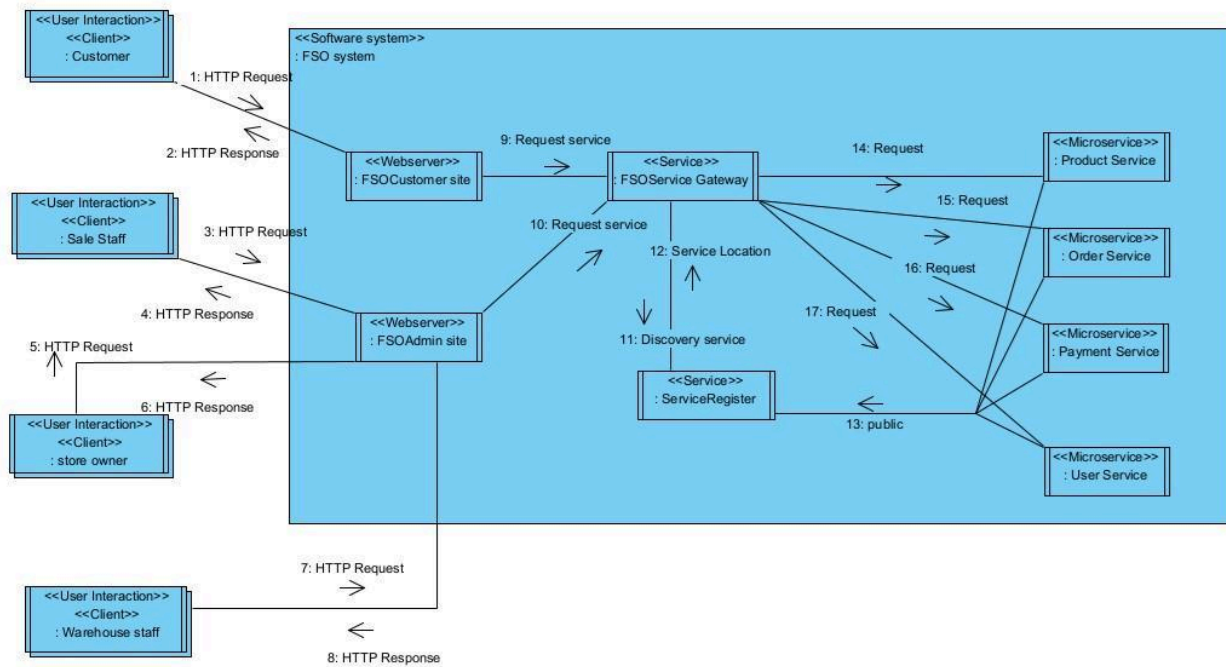


Figure V-6 Component Diagram for the FSO System Architecture Based on Microservice Architecture